My Note of Studying the Quantum Computation

Ye Ning

July，05，2025

目录

1 Introduction

This is a draft of an informal note, a summary of what I read recently about the quantum computing, especially on the topic of so-called Quantum Variational Eigensolver (QVE). In this note, I included not only some discussions over the material I read, but also my own thoughts and comments even if they are immature. I plan to mark the unanswered questions in this note if there is any. Python code is presented here in form of listing.

For now, this note is not organized logically, some material and idea are included in an ad hoc way. But I believe, given more time, with the accumulation of reading and discussion, this note will become informative, formal and satisfying. Hopefully, I wish this note can be eventually developed into a self-contained, well-organized reference book for the future researchers and/or lecturers.

2 The Variational Quantum Eigensolver (VQE)

2.1 Overview of VQE

VQE [9, 11, 18] is an algorithm that uses classical and quantum computing in conjunction to accomplish a task. Usually, the VQE is composed of four parts: an operator, an "ansatz", an estimator and a classical optimizer. The operator is often a Hamiltonian $\mathcal{H}$, that describes the property of the system we are studying. We need to use the variational method to figure out the eigenvector of the operator to minimize the eigenvalue. The ansatz is actually a quantum circuit that prepares a quantum state approximating the eigenvector we are seeking. The gates in the ansatz are parametrized, and we can vary those parameters iteratively to achieve the minimum expection value. The estimator is a means of estimating the expectation value of the opertor $\mathcal{H}$ over the current quantum state. In the following question we are dealing with (the quantum chemical computation see section 6), the ultimalte result that I care about is the expectiona value, which is related to the eigenvaluees (energy level) of the system, and this expectation value is also called the cost function.

Before the invention of VQE, the quantum approach to eigenvalue problem is quantum phase estimation (QPE). In this algorithm, the eigenvector $|\psi\rangle$ of a Hermitian operator $\mathcal{H}$ is regarded as input and the problem is to calculate the corresponding eigenvalue λ, which is determined by successive application of $e^{-i\mathcal{H}t}$ onto $|\psi\rangle$. However, this method needs millions or billions of quantum gates for practical application while only tens to hundreds of gates are available right now.

To overcome this difficulty, VQE is introduced to solve the eigenvalue problem by figuring out the $|\psi\rangle$ that can minimizes the so-called Rayleigh-Ritz quotient:

$$\frac{\langle\psi|\,\mathcal{H}\,|\psi\rangle}{\langle\psi\,|\psi\rangle}, \tag{1}$$

via a variational method. To realize the quantum computation, the Hamiltonian can be written into the second quantization form:

$$\mathcal{H} = \sum_{p,q} h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum_{p,q,r,s} h_{pqrs} a_p^\dagger a_q^\dagger a_r a_s, \tag{2}$$

where $a_p^\dagger$ and a_p are fermionic creation and annihilation operators, respectively. The real numbers h's are one- or two-electron integrals that can be easily computed on a classical computer. Then this formulation should be further mapped onto a quantum computer to calculate

$$\mathcal{H} = \sum_{i\alpha} h_\alpha^i \sigma_\alpha^i + \sum_{ij\alpha\beta} h_{\alpha\beta}^{ij} \sigma_\alpha^i \sigma_\beta^j + \dots. \tag{3}$$

Here Latin indices identify the subsystem on which the operator acts, and Greek indices identify different type of Pauli operators. For now, there are three different mapping schemes, and the mapping process will be discussed further below. Regardless the choice of mapping method, the equation above sometimes is written as

$$\mathcal{H} = \sum_j \alpha_j P_j = \sum_j \alpha_j \prod_\alpha \sigma_\alpha^j, \tag{4}$$

where α_j are real scalar coefficients depend on the hs, and P_j are called Pauli words (strings), representing a product of Pauli matrices. Based on the equation (3), the expectation value of $\mathcal{H}$ is expressed as:

$$\langle \mathcal{H} \rangle = \sum_{i\alpha} \langle \sigma_\alpha^i \rangle + \sum_{ij\alpha\beta} \langle \sigma_\alpha^i \sigma_\beta^j \rangle + \dots. \tag{5}$$

The following snippet of Python code is from [9], demonstrating how the operator is presented as code:

Listing 1: Constructing an Operator for VQE

```
from qiskit.quantum_info import SparsePauliOp

hamiltonian = SparsePauliOp(
    [
        "IIII",
        "IIIZ",
        "IZII",
        "IIZI",
        "ZIII",
        "IZIZ",
        "IIZZ",
        "ZIIZ",
        "IZZI",
        "ZZII",
        "ZIZI",
        "YYYY",
        "XXYY",
        "YYXX",
        "XXXX",
    ],
```

```
    coeffs=[
        -0.09820182 + 0.0j,
        -0.1740751 + 0.0j,
        -0.1740751 + 0.0j,
        0.2242933 + 0.0j,
        0.2242933 + 0.0j,
        0.16891402 + 0.0j,
        0.1210099 + 0.0j,
        0.16631441 + 0.0j,
        0.16631441 + 0.0j,
        0.1210099 + 0.0j,
        0.17504456 + 0.0j,
        0.04530451 + 0.0j,
        0.04530451 + 0.0j,
        0.04530451 + 0.0j,
        0.04530451 + 0.0j,
    ],
)
```

The listing above describes a 16×16 Hamiltonian matrix. The reason is that the length of each Pauli string, such as IIII, or YYYY, is four. For a n-bit Hilbert space, its dimension is 2^n. Therefore, for $n = 4$, its dimensionality is $2^4 = 16$. Besides, in the SparsePauliOp class, those Pauli strings and coefficients are strictly one-to-one related. Take the second Pauli string IIIZ as an example, its corresponding coefficient is -0.1740751. Considering IIIZ is a 4 qubits Pauli tensor product:

$$"IIIZ" = I \otimes I \otimes I \otimes Z,$$

this Pauli string/coefficient pair denotes $-0.1740751 \cdot (I \otimes I \otimes I \otimes Z)$.

In [9], we can also read of the comments of the listing above like:

> Note that in the Hamiltonian above, there are terms like ZZII and YYYY that do not commute with each other. That is, to evaluate ZZII, we would need to measure the Pauli Z operator on qubit 3 (among other measurements). But to evaluate YYYY, we need to measure the Pauli Y operator on that same qubit, qubit 3. There is an uncertainty relation between Y and Z operators on the same qubit; we cannot measure both of those operators at the same time.

In this quotation, we can see there is a problem concerning the measuements of the uncommutative operators. So, how to solve the problem? The answer to this question is: measurement in batches. That means we do not perform the measurements simmultaneous, instead, in every batch of measurement the quantum state will be appropriately rotated to Z basis if necessary, since quantum computer can only measure Z basis. For example, if we want to measure Y, we must rotate Y into Z:

$$Y = R_x\left(-\frac{\pi}{2}\right) Z R_x\left(\frac{\pi}{2}\right).$$

After measured upon Z, the result will be interperated as measurement upon Y. This method does not vilote the priciple of uncertainty, since we do not perform the measurement simultaneously. Instead, for each batch of the measurement, we prepare the same input state. To realize the batch measurement efficiently, we need to catogarize the Pauli strings wisely to decrease the times of measuring. Since it is more like an engineering problem, details of this issue will not be mentioned here.

Now, let's learn something about ansatz. In [9], we are given an ansatz like the figure below. If we want to know the functionality of this circuit, we need to read it column by column like

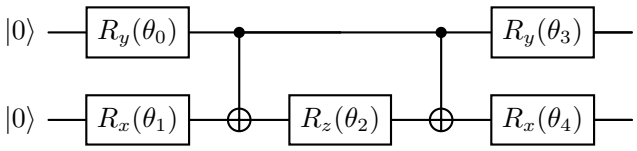

图 1: Five-Step Ansatz

we are reading a piano music staff. Suppose those two lines corresponging to qubits, qubit 0 is the upper one and the qubit 1 is the lower one. Both of them are initialized as $|0\rangle$ From the left to the right, the first column is

$$U_1 = R_y^{(0)}(\theta_0) \otimes R_x^{(1)}(\theta_1);$$

the second column is only a CNOT, which can be expressed as:

$$U_2 = \mathrm{CNOT}_{0\to 1};$$

The unitary operator for the third column is

$$U_3 = I^(0) \otimes R_z^{(1)}(\theta_2);$$

and it is obvious that $U_4 = U_2 = \mathrm{CNOT}_{0\to 1}$. For column 5, its unitary operator is

$$U_5 = R_y^{(0)}(\theta_3) \otimes R_x^{(1)}(\theta_4).$$

Therefore, the total unitary operator of the circuit is

$$U(\theta) = U_5 U_4 U_3 U_2 U_1. \tag{6}$$

2.2 ADAPT-VQE

ADAPT-VQE is a smart greedy algorithm. However, it constructs the ansatz by including the exciting operators (single-excitation, double-excitation) in a term-by-term manner (organically growth). For each step of including a new term, the purpose is to maximize the energy drop (energy gradient), regardless whether if it is a violation of the symmetry of the system. As a result, it can give rise to a low-energy but non-physical state, such as violating the conservation of particle number or spin symmetry.

2.3 Valence Bonding

The first formula in [5] is like

$$|H_2\rangle = \cos(\frac{\theta}{2})|\sigma\bar{\sigma}\rangle + \sin(\frac{\theta}{2})|\sigma^*\bar{\sigma}^*\rangle, \tag{7}$$

where σ means bonding and σ^* means anti-bondig orbital. The bar used here means that the electron in this orbital is spin-down. Therefore, $|\sigma\bar{\sigma}\rangle$ denotes a pair of bonding orbital electrons, one spin-up, the other spin-down. In other words, those symbols are used to clearly label in which orbit the electron is in and its spin direction. In this formula, $\cos(\frac{\theta}{2})$ and $\sin(\frac{\theta}{2})$ are two important parameterized coefficients, as they reflect **the proportionality of quantum superposition** of the bonding orbital states and antibonding obital states. In other words, those two parameters control whether the electrons tend to the "stable combination" or to the "exited combination" of the chemical bonds. $|\sigma\bar{\sigma}\rangle$ denote that the two electrons are in the σ orbital (bond-forming orbital), forming the stable covalence bond. Meanwhile, $|\sigma^*\bar{\sigma}^*\rangle$ is an unstable state. The superposition of those two states gives rise to a more flexible and adjustable wavefunction. The reason that we use the single θ rather than two independent parameters is that by doing so we can guarantee the automatic normalization of the wave functions, which is $\cos^2\frac{\theta}{2} + \sin^2\frac{\theta}{2} = 1$. This is a quantum-circuit-friendly expression which means, it can be easily mapped into a rotation gate.

For a single qubit, the application of the rotation gate on a $|0\rangle$ is as follows:

Listing 2: Sample Python Code

```
from qiskit import QuantumCircuit
from qiskit.circuit.library import RYGate
```

```
theta = 1.2  # Define angle
qc = QuantumCircuit(1)
qc.ry(theta, 0)  # Or use: qc.append(RYGate(theta), [0])
qc.draw()
```

$$|0\rangle \text{ —} \boxed{RY(\theta)} \text{ —}, \tag{8}$$

and matrix form of the single qubit rotation gate is:

$$R_Y(\theta) = \begin{bmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix} \tag{9}$$

Therefore, assume the initial state of a qubit is $|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, then after the rotation via the gate, it turns out to be

$$|\psi\rangle = R_Y(\theta)\,|0\rangle = \begin{bmatrix} \cos(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) \end{bmatrix} = \cos(\frac{\theta}{2})\,|0\rangle + \sin(\frac{\theta}{2})\,|1\rangle\,, \tag{10}$$

which is exactly the wavefunction of a hydrogen molecule.

The following formula is Eq. (4) in [5].

$$\begin{array}{l} |0\rangle \text{ —} \boxed{RY(\theta_0)} \text{ —}\bullet\text{ —} \\ |0\rangle \text{ —} \boxed{RY(\theta_1)} \text{ —}\oplus\text{ —} \end{array} \tag{11}$$

Its corresponding Qiskit should be:

Listing 3: Sample Python Code

```
from qiskit import QuantumCircuit
import numpy as np

theta = 1.2  # 可任意调整角度
qc = QuantumCircuit(2)
qc.ry(theta, 0)  #     =
qc.ry(np.pi, 1)  #     =
qc.cx(0, 1)
qc.draw('mpl')
```

Functionality of each part:

- $R_Y(\theta)$: Exerting Y-axis rotation on the first qubit, constructing the linear superposition of $|\sigma\rangle$ and $|\sigma^*\rangle$.
- $R_Y(\pi)$: rotating the second qubit to its ground state $|1\rangle$, corresponding to the spin-down electron.
- $CNOT(0 \to 1)$: introducing the entanglement, forming the **resonant mixed state** rather than the simple multiplication state of the electron, which is exactly the superposition relation reflected in Eq. 7.

2.4 Mapping Methods

In the following three subsections, we briefly describe an essential process in quantum computation – mapping. Actually, the process is one of the fundamental difference between the quantum computation and the classical computation. Of course, this process requires extra over-head or even some sophisticated algorithms.

First, we need to explain why we need this procedure. Unlike the classical system, qubit cannot be read without wavefunction collapse. For this reason, "parity" should be coded rather than being stored. It should manifest itself naturally in the state evolvement process. For the same reason, keeping the anti-commutativity of the fermionic operators must be a "measurement-free" structural operation, requiring the automatic change of the sign. This requirement can only be achieved by the dynamic construction process of the qubit gates, not the afterward correction by classical bits. In addition, the advantage of quantum computation is essentially based on quantum entanglement and quantum superposition, while in many scenarios, parity is tightly related to the entangled structure. Therefore, parity, as an important quantum feature, is not a certain value that can be stored by a classical bit. For example, in the following superposition state,

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0100\rangle + |1011\rangle), \tag{12}$$

one cannot tell electron is in which orbital exactly, since the coexistence of the two states. Parity is decided by the "number of the electrons in certain interval", and this number can only be obtained from measurement after the collapse of the wavefunction.

Meanwhile, in the framework of second quantization, a fermionic orbit can be either vacant or occupied, similar to the classical bit (0 or 1). As we know, the qubit can only be either $|1\rangle$ or $|0\rangle$ or their superposition, therefore, it is nature to use a qubit to represent a fermionic orbit.

This in turn makes it feasible to map a fermionic Hamiltonian into a qubit Hamiltonian, hence the algorithms like VQE or QPE.

To this end, we define three types of set, and one of the important tasks of mapping schemes is to update those sets correctly:

表 1: Three Key Sets in Bravyi–Kitaev Mapping

Set	**Description**
Update set	Qubits that must be flipped when a mode changes.
Parity set	Tracks parity of modes before the current one.
Occupation set	Tracks whether a mode is occupied.

2.4.1 Jordan-Wigner Mapping (JWM)

Jordan-Wigner mapping is used to convert Fermion operator into spin operator such as Pauli matrix. At first, it was established to solve 1-D spin chain model, but now it is widely used in quantum chemistry and quantum physics simulation. Its key idea is to map each orbital in the Fermion system to a qubit, and use Pauli matrix to represent Fermion operator, without changing the anti-commute property. Here is a simplified version of mapping formula:

$$a_j^\dagger = \left(\prod_{k=0}^{j-1} Z_k \right) \cdot \frac{X_j - iY_j}{2}, \tag{13}$$

where X_j, Y_j, Z_j are the j^{th} qubit on the Pauli matrix. $\prod_{k=0}^{j-1} Z_k$ is also called the **Z chain**, its functionality is to guarantee that when we exchange the Fermions, a negative sign can be correctly introduced so that the overall anti-commutativity is maintained. Please note, in the equation above, when $k = 0$, we follow the rule:

$$\prod_{\text{empty product}} = 1. \tag{14}$$

The anti-commutation relations are:

$$\begin{aligned} \{a_i, a_j\} &= a_i a_j + a_j a_i \\ \{a_i^\dagger, a_j^\dagger\} &= 0 \\ \{a_i, a_j^\dagger\} &= \delta_{ij} \end{aligned} \tag{15}$$

When we are going to map the fermionic operator to the Pauli matrix, we need to keep the anti-commutativity structure. In Jordan-Wigner mapping, the operator on the j^{th} orbital can be expressed as:

- Creation Operator:

$$a_j^\dagger = \left(\prod_{k=0}^{j-1} Z_k\right) \cdot \frac{X_j - iY_j}{2} \tag{16}$$

- Annihilation Operator:

$$a_j = \left(\prod_{k=0}^{j-1} Z_k\right) \cdot \frac{X_j + iY_j}{2}, \tag{17}$$

It is obvious that when the system is complex enough, the Z chain will be very long which causes a deep circuit. Luckily, this in NOT the only way to maintain the overall anti-commutativity. There are two well-known methods can address the anti-commutativity more efficiently. They are **Parity Mapping** and **Bravyi-Kitaev Mapping**.

2.4.2 Parity Mapping

Rather than "memorizing how may Fermion are there on the left side of the orbital", the Parity Mapping method records the "Parity" directly. Its creation operator is like:

$$a_j^\dagger = \left(\prod_{k \in P(j)} Z_k\right) \cdot \frac{X_j - iY_j}{2}, \tag{18}$$

where $P(j)$ is the orbital collection related to the j^{th} orbital, used to guarantee the anti-commutativity. The advantage of this idea is this mapping method does not need the Z chain get longer with the increment of the orbital number. On the contrary, it may become shorter. It can keep the symmetrical property of the system better under some circumstances, such as the projection of spins. However, this mapping strategy is not so straightforward as the JWM, so that the Parity Mapping process is more complicated and harder to debug than JWM.

2.4.3 Bravyi-Kitaev Mapping (BKM)

This is a more sophisticated mapping method, which distribute information to many qubits. It can track down the **local occupation** and the **accumulation of parity** at the same time. It uses a balanced way to encode the anti-commutativity into the circuit to achieve the following two competing goals:

- **Locality**: minimizing how many qubits a single fermionic operator affects.
- **Efficiency**: reducing the number of quantum gates needed to simulate fermionic interactions.

To achieve those goals, **binary tree structure** is used to encode both **occupation** and **parity** information, allowing it to update fewer qubits when simulating fermionic creation/annihilation operators.

Specifically, BKM updates three key sets listed in Table 1 via a transformation matrix, and this transformation matrix is set up in a recursive manner. This leads to a more compact and efficient representation of the system's Hamiltonian.

In this method, we need to generate a so-called Bravyi-Kitaev basis ($\vec{b}_n$) from the occupation state vector ($\vec{f}_n$) via a square transformation matrix β_n such that $\beta_n \vec{f}_n = \vec{b}_n$. The matrix β_n is constructed in a recursive manner:

$$\beta_{2^x} = \left[\begin{array}{c|c} \beta_{2^{x-1}} & 0 \\ \hline 0 & \\ \leftarrow 1 \rightarrow & \beta_{2^{x-1}} \end{array}\right] \qquad \text{where } \beta_1 = (1) \tag{19}$$

Let's repeate the example in [21], for eight qubits, $\beta_n \vec{f}_n = \vec{b}_n$ is specified into (all sums in mod(2)):

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \end{bmatrix} = \begin{bmatrix} f_0 \\ f_1 + f_0 \\ f_2 \\ f_3 + f_2 + f_1 + f_0 \\ f_4 \\ f_5 + f_4 \\ f_6 \\ f_7 + f_6 + f_5 + f_4 + f_3 + f_2 + f_1 + f_0 \end{bmatrix}. \tag{20}$$

From the right side of Eq. 20, we can figure out the rules for constructing Update Set. If the occupation state of the first orbit is changed, then those need to be updated components of $\vec{b}_n$ are b_0, b_1, b_3 and b_7, not all of them. Similarly, when f_1 is changed, the Update Set contains only b_1, b_3 and b_7. We don't need to consider components with even label in $\vec{b}_n$, since those items contain only the occupation state of that orbital itself, and in the Update Set, the orbital itself is NOT included. If we want to construct Update Set $U(j)$, then b_0 to b_j need not be checked, since those components do not contain the information of orbital j. The reason is the transformation matrix is a lower triangular matrix. For example, if f_3 is changed, b_0, b_1 and b_2 will not be affected.

Given the $\vec{f}_n$, to figure out the Parity Set is not a difficult job, as long as we make use of

matrix

$$[\pi_n]_{ij} = \begin{cases} 1, & \text{if } i < j \\ 0, & \text{otherwise} \end{cases}. \tag{21}$$

Then $\pi_n \vec{f}_n$ will give us the Parity Set.

The analysis above tells us if we have already known the $\vec{b}_n$, then we can obtain occupation bais by $\beta_n^{-1}\vec{b}_n = \vec{f}_n$, and by multiplying $\pi_n\beta_n^{-1}$ on the Bravyi-Kitaev basis we can find out the parity.

Once again, using eight qubits as an example, we can see

$$\beta_8^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}. \tag{22}$$

And again, this reverse matrix can be constructed from a recursive relation:

$$\beta_{2^x} = \left[\begin{array}{c|c} \beta_{2^{x-1}}^{-1} & 0 \\ \hline 0_1 & \beta_{2^{x-1}}^{-1} \end{array}\right] \qquad \text{where } \beta_1^{-1} = [1] \tag{23}$$

Reflection: *Although the discussion above shows the effectiveness of the one-orbit-to-one-qubit mapping method, it may not be the only solution to current situation, and it is potentially problematic. First, it suffers from "over-abstraction", which means after the complicated electron wave functions are compressed into binary occupation states, the information about symmetry, entanglement and group structure may be lost. What make things worse is the risk of generating non-physical states, such as the state that violate the conservation of spin, etc. Many body wave function is more than a simple combination of the orbital occupation, it also contains coupling and correlation between the states, which cannot be covered perfectly only by qubits.*

To overcome those defects, people are thinking of other solutions. For example, Tensor Network Quantum Simulation (TNQS) [1] does not use "orbit occupation" as the fundemental block, instead, it uses an entangled structure to describe the whole wavefunction. Group-equivariant Ansatz (GEA) uses group structure to construct variational state. Quantum Natural

Orbitals plus Subspace Encoding method makes use of classical calculation to obtain the principle orbitals and then encode the subspace state. Bosonic or Fermionic Simulation Framework skip over the step of orbital mapping, it uses qubit to denote the vector or the symmetrical spatial wavefunction.

2.5 Measuring the Qubit Circuit

Suppose we have

$$\langle H \rangle = \sum_i h_i \langle \psi | P_i | \psi \rangle , \tag{24}$$

where P_i is the Pauli words like $X_0Y_1Z_2$, the h_i is the corresponding real coefficient, and $|\psi\rangle$ is the current output ansatz quantum state. Then the question is how to measure $\langle \psi | P_i | \psi \rangle$ in the qubit circuit?

Now, take Pauli words $P_i = Z_0Z_1$ as an example, we need to take the following steps to measure:

1. **Preparing the state**: Use the variational circuit to generate $|\psi\rangle = U(\bar{\theta}) |0 \dots 0\rangle$.

2. **Choosing the measuring base**: For example, if we want to measure Z_0Z_1, we need to perform Z base measurement on the $0th$ and the $1st$ qubit, which is the default measurement. Once we get two results $m_0, m_1 \in \{+1, -1\}$, we calculate product $m_0 \cdot m_1$. Except the default measurement, to perform other types of measurements, we need to add base conversion gate before the measurement:

表 2: Base Conversion Gate

The target Pauli word	The corresponding gate to be added
X	Hadamard gate
Y	$S^\dagger$+ Hadamard gate
Z	Don't need to add any gate

3. **Repeating measurement and calculating the average value**: We need to repeat the measurement of P_i many times to get many samples (shots), say 1024 times, to obtain average value of product:

$$\langle P_i \rangle \approx \frac{1}{N} \sum_{j=1}^{N} m_0^{(j)} \cdot m_1^{(j)} \cdot \dots , \tag{25}$$

then we multiply this by h_i to obtain the contribution to the energy from this term.

The relevant Qiskit code can be found in Listing 4.

2.6 Transverse-Field Ising Model

The **Transverse-Field Ising Model** referred to in [5] is a typical quantum spin model, which is widely used in the study of quantum phase transition, entanglement, and the design of the variational algorithm in the quantum computation.

For the 2-site system, [5] shows that the Hamilton as:

$$\hat{H}_{2-site} = -JZ_0Z_1 - h(X_0 + X_1), \tag{26}$$

where:

- Z_i, X_i are the Pauli-Z and Pauli-X operator on the ith qubit.
- J is the coupling strength between spins (along the Z direction).
- h is the strength of the transverse magnetic field (along the X direction).

The first term, $-JZ_0Z_1$, tends to (anti)align the neighboring spin along the z-direction, describing the spin interaction. The second term, $-h(X_0 + X_1)$, introduces the transverse magnetic field, makes the spin flip along the X-direction, describes the quantum fluctuation. Competition between those two terms gives rise to the quantum phase transition, which means the system changes from the ordered state (spin aligns) to the disordered state (spin flips).

2.7 One-shot Calculation

One-shot calculation means in VQE, a complete calculation is performed upon a certain set of parameters without referring to previous results of those parameters in the course of the "continuous evolvement".

More specifically, in [5], there are two optimization strategies:

1. Default strategy
    - Each calculation starts with a randomly initialized angle;
    - Every parameter, such as length of bond, different model parameters, will be optimized independently.
    - Pros: One can jump to any point in the parametric space without worrying about the history.
    - Cons: It is easy to get stuck in the local minimum, especially in a complex system.

2. Adiabatic strategy

   - Start from a relative easier parameter, such as the balance bonding length.
   - Then change the parameters gradually, each new iteration starts with the optimized parameter from the previous circle.
   - Pros: More stable, unlikely to get stuck in local minimum;
   - Cons: Cannot jump to any point easily, one must "cover the whole path".

3 Use of the Basis

3.1 STO-nG

STO-nG is a short of "**S**late-**T**ype **O**rital with **n** **G**aussians", and the STO-3G is the simplest of the set. Take the STO-3G as an example, its main idea is to use 3 Gaussians to fit an approximate Slater type of orbital. There is a two-fold motivation behind this. One is that the STO is very close to the real atomic orbital shape but needs complicated computation; the other is that the Gaussians are efficient in calculation. Therefore, this scheme balances the physical precision and the computational efficiency.

Mathematically, every atomic orbital (such as $1s$, $2s$ and $2p$) can be expressed as:

$$\phi_{\text{STO-3G}} = \sum_{i=1}^{3} c_i \cdot \phi_{\text{GTO}}(\alpha_i), \tag{27}$$

where c_i is the coefficient of each GTO, α_i is the exponential index of each GTO. All the GTO are the radial wavefunctions centered on the nuclear. Usually, those parameters are obtained via the least squares method.

Taking the hydrogen molecule as an example, we can construct a complete quantum Hamiltonian for it step by step. First, we need to use tools like *Qiskit Nature* or *PySCF* to decide geometry such as internuclear distance, such as 0.74 Å, which is close to ground state. Then the basis STO-3G is chosen. After that, we need to initiate the Hartree-Fock calculation to obtain the single- and double- electron integration. Under STO-3G, two $1s$ orbitals (one for each hydrogen atom) are used. After Hatree-Fock, we get Hamiltonian like:

$$\hat{H} = \sum_{pq} h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} h_{pqrs} a_p^\dagger a_q^\dagger a_r a_s. \tag{28}$$

Therefore, we obtain four (spin) orbitals: 2 spatial orbitals times α, β spin leads to 4 fermionic up/down operator indices. We can now map the Hamilton into Pauli words by using either JWM or BKM. Taking JWM as an example, the mapping leads to:

$$H = c_0 I + c_1 Z_0 + c_2 Z_1 + c_3 Z_0 Z_1 + c_4 Y_0 X_1 X_2 Y_3 + \ldots, \tag{29}$$

in which every term such as $Z_0 Z_1$, $Y_0 X_1 X_2 Y_3$ etc. are measurable Pauli words, and each corresponding to a coefficient c_i. Those coefficients are the result of the collection of molecular integration after the anti-commutative expansions. The related Qiskit code is demonstrated in Listing 5.

Now, let us take a closer look at the STO-3G for hydrogen molecule. As mentioned before, for each of the hydrogen atom, it approximate Slater orbital is:

$$\phi_{\text{STO}} = N e^{-\zeta r}, \tag{30}$$

and in STO-3G, this $1s$ orbital is approximately expressed as a linear comibination of 3 GTO:

$$\phi_{1s}^{\text{STO-3G}}(r) = \sum_{i=1}^{3} c_i \cdot \phi_{\text{GTO}}^{(i)}(r). \tag{31}$$

More specifically, in spherical coordinate, a standard GTO is:

$$\phi_{\text{GTO}}^{(i)}(r, \theta, \phi) = N_i \cdot Y_{l,m}(\theta, \phi) \cdot r^{2n} e^{-\alpha_i r^2}, \tag{32}$$

where N_i is the normalization factor, $Y_{l,m}$ is the spherical harmonic function for the angular part, r^{2n} is the polynomial radial part ($n = 0$ is for the $1s$ orbital), α_i is the i^{th} GTO Gaussian Index. As an example, for the $1s$ orbital of hydrogen, the angular part is $Y_{0,0} = \frac{1}{\sqrt{4\pi}}$, which is a constant, reflecting the fact that the s orbital is isotropic. The radial part is $r^0 = 1$. Therefore, every GTO is simplified into

$$\phi_{\text{GTO}}^{(i)} = N_i \cdot e^{-\alpha_i r^2}. \tag{33}$$

3.2 Hylleraas, the Possible Basis to Form Ansatz

Hylleraas base is a set of non-orthogonal base used in high precision calculation of electron's relation in traditional quantum chemical computation. It is especially good for two-electron system, such as Helium. In this base, inter-electronic distance r_{12} is explicitly introduced to capture the interrelation between the electrons precisely. Therefore, one of its typical wavefunctions is

$$\phi(r_1, r_2, r_{12}) = r_1^n \cdot r_2^m \cdot r_{12}^p \cdot e^{-\zeta(r_1 + r_2)}. \tag{34}$$

Those wavefunctions form the set of Hylleraas basis, which can be linearly combined into

$$\Psi_{H_2} = \sum_{n,m,p} c_{nmp} \cdot r_1^n \cdot r_2^m \cdot r_{12}^p \cdot e^{-\zeta(r_1 + r_2)}, \tag{35}$$

where the coefficient c_{nmp} can be obtained via variation.

To apply Hylleraas base to quantum computational chemistry, the challenges are:

- Hylleraas base is usually not orthogonal, which means it is hard to map the base to the qubit.
- It contains non-polynomial function such as r_{12}, which is not easy to be presented by the standard quantum gate.

After all, the current main-stream quantum mapping such as Jordan-Wigner, Bravyi-Kitaev are based on the orthogonal orbital structures.

However, there is also a great potential of this investigation. If the Hylleraas base can be converted into appropriate quantum gates, it may decrease the complexity of ansatz dramatically. Meanwhile, it is apt to construct few-body high precision ansatz, especially for the 2-electron VQE.

3.3 Comparison Between two Types of Basis

表 3: Comparison Between GTO and Hylleraas

Property	GTO	Hylleraas
Exponential Part	$e^{-\alpha r^2}$	$e^{-\zeta r}$
Polynomial Part	r^l	$r_1^n r_2^m r_{12}^p$
Computational Efficiency	High	Low
Interelectronic Relation	Weak	Strong
Application	Many-electron System	Two-electron System (He, H_2)

The biggest difference between the STO and Hylleraas is the orthogonality, which is defined as

$$\int \phi_i^*(r)\phi_j(r)dr = 0 \quad (\text{when } i \neq j). \tag{36}$$

Orthogonality indicates the spatial independence of the orbital wavefunctions, which means orthogonal basis will not be "mixed up". Orthogonality plays an important role in the quantum circuit mapping. The challenges brought by the non-orthogonal basis may be:

- Creation and anihilation operator are no longer anti-commutative, which means $\{a_p^\dagger, a_p\}$.
- Electron state is no longer standard the tensor product state in Forc space. If the state cannot be expanded clearly, it will be difficult to be represented by the qubit.

To overcome the difficulties listed above, one of the possible solutions is "orthogonization", such as Löwdin method, which, of course, will bring extra complexity in qubit circuits.

4 Quantum Circuit and the Use of Qiskit

4.1 The quantum circuit

4.1.1 The Pauli Matrices

表 4: Collection of Pauli Matrices

Name	Symbol	Matrix Form
Pauli-X	X	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y	Y	$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z	Z	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Identity Matrix	I	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

4.1.2 The CNOT gate

CNOT (Controlled-NOT) gate is the used onto two qubits: one is for controlling, the other is the target bit. Its functionality is:

- **When the control bit is** $|1\rangle$, flip the target value.
- **When the control is** $|0\rangle$, the target is not changed.

Its matrix form is:

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \tag{37}$$

corresponding to the 4D space of two qubits. Together with the Hadamard gate, CNOT can give rise to the entanglement state, known as the **Bell state**.

Howeve, in practice, if the topological structure of a quantum computer is taken into account, the realization of the CNOT from the control qubit to the target may not be straightforward. For example, as mentioned in [11], if qubit a is not connected to qubit c directly,

say qubit b is the bridge qubit connect both of them, like the figure beow: In this scenario,

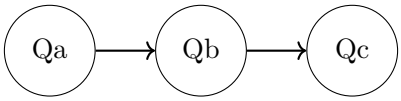

图 2: Connectivity of Three Qubits

$\text{CNOT}_{a\to c}$ cannot be realized directly, it must be realized via qubit b:

$$\text{CNOT}_{a\to c} = \text{CNOT}_{b\to c}\text{CNOT}_{a\to b}\text{CNOT}_{b\to c}\text{CNOT}_{a\to b}. \tag{38}$$

As we can see, the decompostion of $\text{CNOT}_{a\to c}$ is necessary, since we want to change the target qubit c without changing the state of ancilla qubit, which is qubit b here. Actually we can let $|abc\rangle = |110\rangle$ as the initial value of those three qubits, and if we want to obtain $|111\rangle$ via $\text{CNOT}_{a\to c}$, then the decomposition in Eq. 38 is the only way.

4.1.3 The CZ gate

Its matrix form is

$$CZ = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}, \tag{39}$$

and its functionality is:

- when the control qubit is $|1\rangle$, Z gate is exerted to the target qubit, which means the phase is flipped.
- When the control qubit is $|0\rangle$, the target qubit does not change.

Both CZ and CNOT can cause entanglement, and both of them can convert to each other via Hadamard gate:

$$CNOT_{c,t} = (I \otimes H) \cdot CZ_{c,t} \cdot (I \otimes H), \tag{40}$$

which means we can obtain CNOT gate via CZ gate and Hadamard gate.

4.2 The Qiskit

- In Qiskit, the TwoLocal or EfficientSU2 are used to construct parameterized ansatz.
- In Qiskit, all the simulation can be done via AerSimulator to realize testing and optimization.

4.2.1 Collection of Qiskit Code

Listing 4: Circuit Measurement

```
from qiskit.opflow import PauliSumOp
from qiskit import QuantumCircuit, Aer, execute

# 假设你要测 H = 0.7 * ZI + 0.6 * IZ - 0.9 * ZZ
H = PauliSumOp.from_list([('ZI', 0.7), ('IZ', 0.6), ('ZZ', -0.9)])

# ansatz 电路
qc = QuantumCircuit(2)
qc.ry(0.5, 0)
qc.cx(0, 1)

# 添加测量操作（可用 Estimator 接口更高效）
```

Listing 5: Generating Hamilton

```
from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.second_q.mappers import JordanWignerMapper
from qiskit_nature.second_q.problems import ElectronicStructureProblem

# 构建分子对象
driver = PySCFDriver(atom='H␣.0␣.0␣.0;␣H␣.0␣.0␣.74', basis='sto3g')
problem = ElectronicStructureProblem(driver)

# 得到费米子哈密顿量
second_q_op = problem.second_q_ops()['ElectronicEnergy']

# 使用 JW 映射
mapper = JordanWignerMapper()
qubit_op = mapper.map(second_q_op)

print(qubit_op)
```

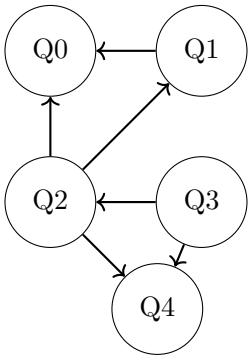

图 3: Connectivity Graph of ibmqx4 Qubits

5 Study of the Common Quantum Computing Algorithms

5.1 Grover's algorithm

Grover's algorthim [11, 16, 7] enables one to find a specific item within a randomly ordered database of N using $O(\sqrt{N})$ operations. This approach can be used to accelerate the search by constructing an "oracle" which is in essence a quantum circuit that can flip the state of an ancillary qubit if the searching criterion is fullfilled. To this end, we need to set up a function $f(\boldsymbol{x})$ will output 1 if the input $\boldsymbol{x}$ equals the target value $\boldsymbol{x}^*$. Therefore, the functio is defined as

$$f(\boldsymbol{x}) = \begin{cases} 1, & \text{if } \boldsymbol{x} = \boldsymbol{x}^* \\ 0, & \text{otherwise} \end{cases}. \tag{41}$$

With this function, we can turn any input into a binary output. For now, the function $f(\boldsymbol{x})$ is only an abstract symbole. Its specific form is decided by the problem we are solving. For example, if we want to find a naive result of $\boldsymbol{x}^*$ such that $x_1^* = 1$ and $x_2^* = 1$ for $\boldsymbol{x} = x_1 x_2$. This essentially means $f(\boldsymbol{x})$ here is an AND gate. But AND is not reversible so that it cannot be a quantum gate. Fortunately, we have Toffoli gate which can take three bits as input and output three bits. The first two bits are used as the control bits so that the third bit flips if the control bits are all 1s. The details of the Toffoli can be found in section 4.1. In other word, for this problem, the orcal quantum circuit can be realized by a Toffoli gate. In general, the functionality of the oracle is

$$|\boldsymbol{x}\rangle |q\rangle \xrightarrow{O} |\boldsymbol{x}\rangle |f(\boldsymbol{x}) \oplus q\rangle , \tag{42}$$

where O is the oracle operator and $\oplus$ denots XOR operation. Based on the oracle opertor O, Grover operator is defined

$$G = (2 |\psi\rangle \langle\psi| - I)O, \tag{43}$$

where $|\psi\rangle = \frac{1}{\sqrt{N}} \sum_i |i\rangle$ is the uniform superpostion over all the basis states and I is the identity operator. Since by now, we have already known the behavior of oracle opertor O, then we need to analyze the purpose of $(2 |\psi\rangle \langle\psi| - I)$. Let use this part of operator onto an arbitary state, given by $\sum_i a_i |i\rangle$. Please note the arbitary state is different from the $|\psi\rangle$ for the previous one does not have equal amplitude for each component. This is important, since the change of the amplitudes will become one of main concerns of this algorithm. Therefore,

$$(2 |\psi\rangle \langle\psi| - I) \sum_i a_i |i\rangle = \sum_i (2\langle a\rangle - a_i) |i\rangle . \tag{44}$$

From the Eq. 44, the effect of the operator on the arbitary state is to flip the amplitude of each component $|i\rangle$ (with the amplitude of a_i) about the mean amplitude $\langle a\rangle$. This property plays

a very important role in this scheme. For this reason, a brief explanation over the expression $2\langle a\rangle - a_i$ is necessary. Suppose, without losing generality, $a_i > \langle a\rangle$, then the distance between the two value is $a_i - \langle a\rangle$. After flipping, the new value is $\langle a\rangle - (a_i - \langle a\rangle)$, hence the result $2\langle a\rangle - a_i$.

Now, it is time to get the initial state ready as shown in the following figure (the first two are the input qubits and the third is the ancillary qubit): After the Hadamard transformation,

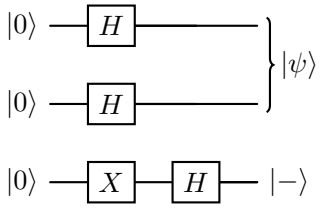

图 4: State Preparation

the two $|0\rangle$s are turned into $|\psi\rangle$ and the ancillary qubit is turned into $\frac{|0\rangle-|1\rangle}{\sqrt{2}}$. So, the prepared state is $|\psi\rangle\frac{|0\rangle-|1\rangle}{\sqrt{2}}$. If the input value is exactly what we are looking for, then the oracle operator will disclose

$$O|\boldsymbol{x}^*\rangle\frac{|0\rangle-|1\rangle}{\sqrt{2}}\rightarrow|\boldsymbol{x}^*\rangle\frac{|f(\boldsymbol{x}^*)\oplus 0\rangle-|f(\boldsymbol{x}^*)\oplus 1\rangle}{\sqrt{2}}=|\boldsymbol{x}^*\rangle\frac{|1\rangle-|0\rangle}{\sqrt{2}}=-|\boldsymbol{x}^*\rangle\frac{|0\rangle-|1\rangle}{\sqrt{2}}. \quad (45)$$

From this equation we can see, if oracle operator is exerted on the target value $|\boldsymbol{x}^*\rangle$, its amplitude will be changed into its oppsite number, while other state will be the same. This will make a big difference between the targeted state and the other states, since the "flipped about the mean amplitude" will be followed. Then we can repeat this procedure many times and the difference will be accumulated until the target state is well-marked. In the worst case, this procedure will be repeated for $\frac{\pi\sqrt{N}}{4}$ times.

5.2 Bernstein-Vazirani Algorithm

BV algorithm [11, 16, 7] is used to solve this type of problems: suppose we have a classical Boolean function $f:\{0,1\}^n\mapsto\{0,1\}$. If there is a *hidden string* $\mathbf{s}$ which is unknow, this Boolean function can always be expressed as $f_{\mathbf{s}}(\mathbf{x})=\oplus_i s_i x_i\equiv\langle\mathbf{s},\mathbf{x}\rangle$ which can be evaluated by an oracle like we saw in Grover's algorithm, then the question is can we find out the hidden string with just a single application of the oracle?

More note on $f:\{0,1\}^n\mapsto\{0,1\}$ and $f_{\mathbf{s}}(\mathbf{x})=\oplus_i s_i x_i\equiv\langle\mathbf{s},\mathbf{x}\rangle$. Here $\{0,1\}^n$ means a set of binary string, and n is the length of the string. For example, when $n=2$, this set contains elements like (00), (01), (10), (11). As to the definition of $\langle\mathbf{s},\mathbf{x}\rangle$, we can understand it via a

specific example. If we have $\mathbf{s} = (1, 0, 1)$ and $\mathbf{x} = (1, 1, 1)$, then the expansion of $\oplus_i s_i x_i$ is $(1 \times 1 + 0 \times 1 + 1 \times 1) \bmod 2$. Two details need to be emphasized here: first, the mod 2 is mandatory; second, the digit in the binary string is the **index** of qubit, not the component of the vector on that qubit. For example, our convention is like $|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. If you have a digital 0 at hand, then this 0 is the 0 on the left side of the equation, not the second component on the right side of the equation.

Similar to Grover's algorithm, we need an oracle U_s to evaluate the function $f_{\mathbf{s}}(\mathbf{x})$. Due to requirement of unitarity and reversiblity, the oracle operator also needs to work on an ancilla qubit $|q\rangle$ [13]. Therefore, we have

$$U_s \left|\mathbf{x}\right\rangle \left|q\right\rangle = \left|\mathbf{x}\right\rangle \left|q \oplus \left\langle \mathbf{s}, \mathbf{x} \right\rangle\right\rangle . \tag{46}$$

Denoting $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$, we can verify from Eq. 45 that,

$$U_s|\mathbf{x}\rangle|-\rangle = (-1)^{\langle \mathbf{s},\mathbf{x}\rangle}|\mathbf{x}\rangle|-\rangle. \tag{47}$$

Meanwhile, the n-qubit Hadamard operator can be expanded in the outer-product form:

$$H^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x},\ \mathbf{y}\in\{0,1\}^n} (-1)^{\langle \mathbf{x},\mathbf{y}\rangle} \left|\mathbf{y}\right\rangle \left\langle \mathbf{x}\right| \tag{48}$$

U_s and $H^{\oplus n}$ are the only two operators used in the BV algorithm, and the process is presented by

$$\begin{aligned} |0\rangle^n |n\rangle \xrightarrow{H^{\otimes(n)}\otimes H} \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x}=0}^{2^n-1} |\mathbf{x}\rangle \otimes |-\rangle \xrightarrow{U_s} \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x}=0}^{2^n-1} (-1)^{\langle \mathbf{s},\mathbf{x}\rangle} |\mathbf{x}\rangle \otimes |-\rangle \\ \xrightarrow{H^{\otimes n}} \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x},\ \mathbf{y}=0}^{2^n-1} (-1)^{\langle \mathbf{s},\mathbf{x}\rangle \oplus \langle \mathbf{x},\mathbf{y}\rangle} |y\rangle \otimes |-\rangle \equiv |\mathbf{s}\rangle \otimes |-\rangle . \end{aligned} \tag{49}$$

5.3 The Quantum Fourier Transform (QFT)

QFT is a core quantum computing algorithm plays an important role in a couple of other algorithms [16]. QFT is based on the classical *discrete Fourier Transform* (DFT), which takes as input a vector of complex numbers, $x_0, ..., x_{N-1}$ where the length N is a fixed parameter. The output of the DFT is another set of complex numbers $y_0, ..., y_{N-1}$ which is related to the input xs by

$$y_k \equiv \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k}/N. \tag{50}$$

Its inverse is given by

$$x_j \equiv \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} y_k e^{-2\pi ijk}/N. \tag{51}$$

In QFT, DFT is done on the *amplitude* of a quantum state

$$\sum_{j=0}^{N-1} x_j \left|j\right\rangle \longrightarrow \sum_{k=0}^{N-1} y_k \left|k\right\rangle, \tag{52}$$

where, as we can see, the y_k are now the DFT of the amplitude of x_j. Such a transform is possible as long as we can find a unitary operator $\hat{F}$ such that

$$\left|\tilde{\psi}\right\rangle = \hat{F}\left|\psi\right\rangle, \quad \text{with} \quad \hat{F}^\dagger \hat{F} = \hat{I}. \tag{53}$$

Similar to Eqn. 48, both $\hat{F}^\dagger$ and $\hat{F}$ can be written into the outer product form:

$$\hat{F}^\dagger = \left(\sum_{j,k} \frac{e^{-2xijk/N}}{\sqrt{N}} \left|j\right\rangle \left\langle k\right|\right), \quad \text{and} \quad \hat{F} = \left(\sum_{j',k'} \frac{e^{2xij'k'/N}}{\sqrt{N}} \left|k'\right\rangle \left\langle j'\right|\right). \tag{54}$$

With this, it is easy to confirm that $\hat{F}^\dagger \hat{F} = \hat{I}$.

Let $N = 2^n$, where n is the number of qubit. The basis $\left|0\right\rangle, ..., \left|2^n - 1\right\rangle$ is the computational basis for those qubits. In other words, we have $\left|j\right\rangle$ with $0 \leq j \leq n-1$. Now, there comes the tricky part, in the upcoming discussion, instead of using decimal number j, we use its corresponding binary form $j_1 j_2 ... j_n$ fulfills $j = j_1 2^{n-1} + j_2 2^{n-2} + \cdots + j_n 2^0$. As for the *binary fraction*, we have $j_l/2 + j_{l+1}/4 + \cdots + j_m/2^{m-l+1}$, we use the notation $0.j_l j_{l+1} \cdots j_m$. With these notations, the transformation

$$\left|j\right\rangle \longrightarrow \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi ijk/N} \left|k\right\rangle. \tag{55}$$

Example 5.1. *For* $\left|0\right\rangle$*, we have*

$$\left|0\right\rangle \rightarrow \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i0k/N} \left|k\right\rangle. \tag{56}$$

Since k *is a binary number, which means* $k \in \{0, 1\}$*, then* $N = 2$*. Eqn. (56) can be further expanded into* $\frac{1}{\sqrt{2}}(\left|0\right\rangle + \left|1\right\rangle)$*. Based on that, it is easy to see*

$$\begin{aligned} \left|00\right\rangle = \left|0\right\rangle\left|0\right\rangle &\rightarrow \frac{1}{2}(\left|0\right\rangle + \left|1\right\rangle) \otimes (\left|0\right\rangle + \left|1\right\rangle) \\ &= \frac{1}{2}(\left|00\right\rangle + \left|10\right\rangle + \left|01\right\rangle + \left|11\right\rangle). \end{aligned} \tag{57}$$

So, the Fourier transform of the n *qubit state* $\left|00...0\right\rangle$ *is* $\frac{1}{2^{n/2}}(\left|00...0\right\rangle + \left|10...0\right\rangle + \cdots + \left|11...1\right\rangle)$

Follow the idea of Example 5.1, Eqn. 55 can be written into the useful *product representation*:

$$\begin{aligned} &|j_1, \cdots, j_n\rangle \rightarrow \\ &\frac{1}{2^{n/2}} \left(|0\rangle + e^{2\pi i 0.j_n} |1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_{n-1} j_n} |1\rangle\right) \otimes \cdots \otimes \left(|0\rangle + e^{2\pi i 0.j_1 j_2 \cdots j_n} |1\rangle\right). \end{aligned} \tag{58}$$

The equivalence between Eqn. 55 and Eqn. 58 can be mathematically proved by

$$\begin{aligned} |j\rangle &\rightarrow \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i j k/2^n} |k\rangle \\ &= \frac{1}{2^{n/2}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} e^{2\pi i j (\sum_{l=1}^{n} k_l 2^{-l})} |k\rangle \\ &= \frac{1}{2^{n/2}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} \bigotimes_{l=1}^{n} e^{2\pi i j k_l 2^{-l}} |k_l\rangle \\ &= \frac{1}{2^{n/2}} \bigotimes_{l=1}^{n} \left[\sum_{k_l=0}^{1} e^{2\pi i j k_l 2^{-l}} |k_l\rangle\right] \\ &= \frac{1}{2^{n/2}} \bigotimes_{l=1}^{n} \left[|0\rangle + e^{2\pi i j 2^{-l}} |1\rangle\right]. \end{aligned} \tag{59}$$

It is easy to see, the last line of Eqn. 59 is equivalent to the right side of Eqn. 58. Now, we can see the significance of Eqn. 58 lies in the fact that the unitary

$$|0, 1\rangle \longrightarrow \frac{1}{\sqrt{2}}(|0\rangle \pm e^{i\theta} |1\rangle) \tag{60}$$

is a Hadamard gate followed by a Z-rotation by θ defined by

$$Z(\theta) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{bmatrix} \tag{61}$$

In Eqn. 58, the rotation depends on the values of the other bits. So we should expect to be able to build the QFT out of *Hadamards* and *controlled-phase rotation gates*. Define the rotation

$$\hat{R}_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}. \tag{62}$$

The controlled-$\hat{R}_k$ gate performs if and only if a control bit is $|1\rangle$ rather than $|0\rangle$. Putting those together, we can get figure 5. In the first line of this figure, after applying the Hadamard gate to the first bit produces the state

$$\frac{1}{\sqrt{2}} \left(|0\rangle + e^{2\pi i 0.j_1}\right) |j_2, \cdots, j_n\rangle. \tag{63}$$

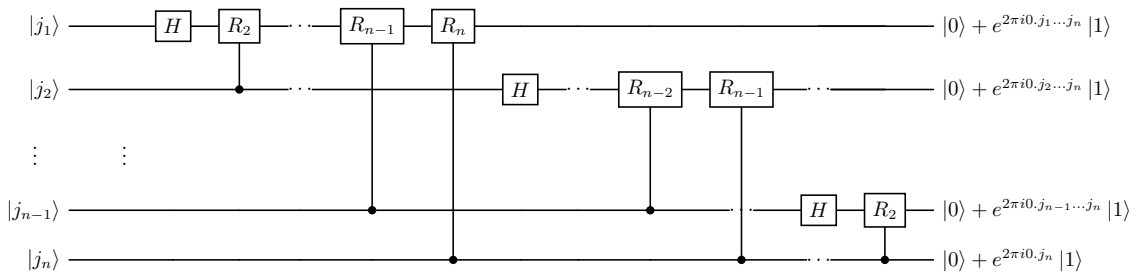

图 5: Circuit of Quantum Fourier Transform

This process continues for the rest of the qubit, giving the final state

$$\frac{1}{2^{n/2}} \left(|0\rangle + e^{2\pi i 0.j_1 j_2 \cdots j_n} |1\rangle\right) \otimes \left(|0\rangle + e^{i\pi i 0.j_2 \cdots j_n} |1\rangle\right) \otimes \cdots \otimes \left(|0\rangle + e^{2\pi i 0.j_n}\right). \tag{64}$$

Compare Eqn. 64 and Eqn. 58, we can see swap operations are needed to reach the desired result. Therefore, it is time for us to count the total number of the gates needed in this circuit. For the first qubit, there are one Hadamard gate and $n-1$ conditional rotation – a total of n gates. Then, for the second qubit, one Hadmard gate plus $n-2$ conditional rotations leads to $n-1$ gates. So we need $n + (n-1)$ gates for the first two qubits. This counting proceeds for the rest ends up with $n(n+1)/2$ total number of gates. Don't forget we also need to take into account $n/2$ swaps, each of which need three controlled-NOT gates. Therefore, this circuit provides a $\Theta(n^2)$ algorithm for pefroming the quantum Fourier Transform. Compared to the best classical algorithm for *Fast Fourier Transform* (FFT) requirement of $\Theta(n2^n)$, the quantum computing is terrific. However, it is still unknown for now whether the QFT can be used to real problem directly to speed up the computation as commemnted in [16].

> "The problem is that the amplitude in a quantum computer cannot be directly accessed by measurement. Thus, there is no way of determining the Fourier transfomed amplitudes of the original state. Worse still, there is in general no way to efficiently prepare the original state to be Fourier transfomed. Thus, finding uses for the QFT is more subtle than we might have hoped."

5.4 Phase Estimation

Phase Estimation (PE) [11, 16] is one of the scenarios that the QFT can be used, and PE itself, as a quantum algorithm, can serve as a subroutine of other quantum computing

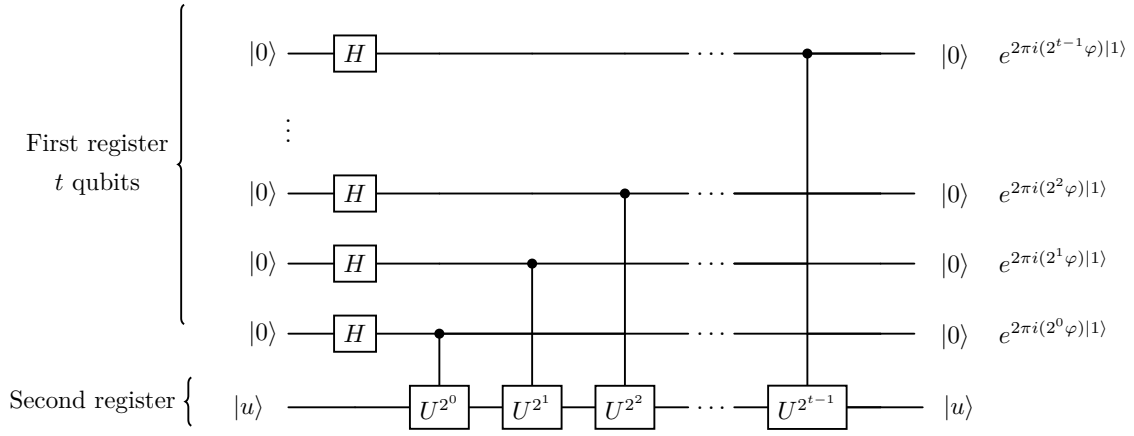

图 6: The First Stage of PE

application. PE is designed for solving a certain type of eigenvalue problem where a unitary operator U has an eigenvector $|u\rangle$ with eigenvalue $e^{2\pi i\varphi}$, with unknown value of the φ.

To establish a complete PE, we first need an oracle (black box) to help us to prepare the state $|u\rangle$ and performing the controlled U^{2^j} operation, where j is a non-negative integer. After that, PE needs two type of register. One contains t qubits initially in the state $|0\rangle$, the other is used to store $|u\rangle$. Based on the established hardware, PE can perform the following three steps:

1. Circuit shown in Fig 6 will be applied;
2. *Inverse* QFT will be performed on the first register;
3. Read out the state of the first register by doing a measurement in the computational basis.

As we can see in Fig. 6, this circuit starts by applying the Hadamard on the first register and then followed by application of controlled-U operators on the second register. The result of this circuit is

$$\frac{1}{2^{t/2}}\left(|0\rangle + e^{2\pi i 2^{t-1}\varphi}|1\rangle\right)\otimes\left(|0\rangle + e^{2\pi i 2^{t-2}\varphi}|1\rangle\right)\cdots\left(|0\rangle + e^{2\pi i 2^{0}\varphi}\right) = \frac{1}{2^{t/2}}\sum_{k=0}^{2^t-1} e^{2\pi i\varphi k}|k\rangle\,. \quad (65)$$

If we expand φ into its t binary bit form and use the notation $\varphi = 0.\varphi_1\cdots\varphi_t$, the left side of

Eqn. 65 can be rewritten into

$$\frac{1}{2^{t/2}}\left(|0\rangle + e^{2\pi i 0.\varphi_1}|1\rangle\right)\otimes\left(|0\rangle + e^{2\pi i 0.\varphi_{t-1}\varphi_t}|1\rangle\right)\cdots\left(|0\rangle + e^{2\pi i 0.\varphi_1\varphi_2\cdots\varphi_t}|1\rangle\right), \tag{66}$$

which is exactly the same as Eqn. (58). Therefore, the second step of PE is to apply the inverse QFT, which gives rise to the product state $|\varphi_1\cdots\varphi_t\rangle$. The measurement will tell us the exact value of φ. This is an ideal scenario when φ can be expressed as a t bit binary expansion. However, even if this condition is not fully fullfilled, it turns out that the procedure will produce a pretty good approximation with high probability.

5.5 HHL Algorithm for Linear Systems Problem

Linear system problem can be met everywhere. To solve it on a quantum computer, we need to make use of the QFT and PE analyzed above. This problem is defined as

$$A|x\rangle = |b\rangle, \tag{67}$$

where A is a Hermitian matrix and both solution to be found $|x\rangle$ and the constant vector $|b\rangle$ have already been normalized, which means $\|\vec{x}\| = \left\|\vec{b}\right\| = 1$. The solution can be expressed as $|x\rangle = \frac{A^{-1}|b\rangle}{\|A^{-1}|b\rangle\|}$. When $\|A^{-1}|b\rangle\| = 1$, this expression can be further simplified as

$$|x\rangle = A^{-1}|b\rangle \tag{68}$$

The quantum algorithm for the linear system was first proposeb by Harrow, Hassidim, and Lloyd, and hence the short name HHL. Mathematical formulation can be summarized briefly below. Using spectral decompostion theorem, operator A can be expressed as

$$A = \sum_{j=0}^{N-1}\lambda_j|u_j\rangle\langle u_j|, \tag{69}$$

where λ_j is the j^{th} eigenvalue of matrix A. Since A is Hermitian, its eigenvectors can form orthonormal basis, and its inverse can be expressed as

$$A^{-1} = \sum_{j=0}^{N-1}\lambda_j^{-1}|u_j\rangle\langle u_j|. \tag{70}$$

Now, let's express the constant vector $|b\rangle$ in terms of eigenbasis of A:

$$|b\rangle = \sum_{j=0}^{N-1} b_j|u_j\rangle. \tag{71}$$

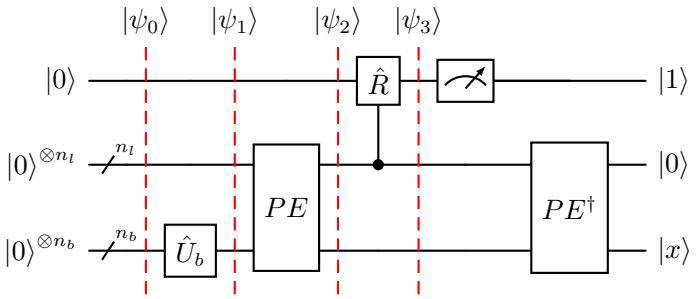

图 7: Circuit of HHL

Together with Eqn. 68 and Eqn. 70, we can see

$$\begin{aligned} |x\rangle &= \left(\sum_{j=0}^{N-1} \lambda_j^{-1} |u_j\rangle \langle u_j|\right) \left(\sum_{i=0}^{N-1} b_i |u_i\rangle\right) \\ &= \sum_{j=0}^{N-1} \sum_{i=0}^{N-1} \lambda_j^{-1} b_i |u_j\rangle \langle u_j|u_i\rangle \\ &= \sum_{j=0}^{N-1} \lambda_j^{-1} b_j |u_j\rangle . \end{aligned} \tag{72}$$

It is apparent from the Eqn. 72 that if we want to figure out $|x\rangle$, we need to know the knowledge of the eigenvalues of A, which can be obtained from the PE algorithm above. Therefore, we have the circuit below to help us do the job.

5.6 Sample-based Quantum Diagonalization of a Chemistry Hamiltonian

Sample-based quantum diagonalization (SQD) algorithm [19, 10] is a method for finding eigenvalues and eigenvectors of quantum opertors, using quantum and distributed classical computing together. Classical distributed computing is used to process samples obtained from a quantum processor, and to project and diagonalize a target Hamiltonian in a subspace spanned by them. SQD is known to work well when the target eigenstate is sparse.

In 2.4, we have learned in quantum chemistry the formalism of second quantization can be used to describe the electronic structure of a molecule. Upon the definition of annihilation operator and creation operator, we also defined their anti-commutation relations in Eqn. 15. However, in this construction, the spin, which is an important feature of electron, was not taken into account. To compensate that, we introduce lables $\hat{\alpha}_{p\sigma}$, where σ is used to include either

spin-up state α or spin-down state β. Accordingly, Eqn 2 is updated into

$$\hat{H} = \sum_{pr\sigma} h_{pr} \hat{a}^\dagger_{p\sigma} \hat{a}_{r\sigma} + \frac{1}{2} \sum_{prqs\sigma\tau} h_{prqs} \hat{a}^\dagger_{p\sigma} \hat{a}^\dagger_{q\tau} \hat{a}_{s\tau} \hat{a}_{r\sigma}, \tag{73}$$

where again h_{pr} and h_{prqs} are complex numbers from molecular integrals.

5.6.1 Unitary Cluster Jastrow (UCJ) ansatz

SQD requires a quantum circuit ansatz to draw samples from Local UCJ (LUCJ) [15], due to its combination of physics and hardware friendliness, is adopted here. This ansatz is based on UCJ [12] which was first proposed by Robert Jastrow to solve many-body problem with strong forces. To address the short range strong force problem, he introduced a product of exponentialized wavefunctions containing a parameter that can be used in variational computation to reach the minimum energy of a quantum system. This wavefunction, which is later called unitary cluster Jastrow (UCJ), can fit into the current quantum computing to solve quantum chemistry problem. It looks like

$$|\psi\rangle = \prod_{\mu=1}^{L} e^{\hat{K}_\mu} e^{i\hat{J}_\mu} e^{-\hat{K}_\mu} |\Phi_0\rangle, \tag{74}$$

where $|\Phi_0\rangle$ is a reference state, often taken to be the Hartree-Fock state, and the $\hat{K}_\mu$ and $\hat{J}_\mu$ are defined as [10, 15]

$$\hat{K}_\mu = \sum_{pq,\sigma} K^\mu_{pq} \hat{a}^\dagger_{p\sigma} \hat{a}_{q\sigma}, \; \hat{J}_\mu = \sum_{pq,\sigma\tau} J^\mu_{pq,\sigma\tau} \hat{n}_{p\sigma} \hat{n}_{q\tau}, \tag{75}$$

where $\hat{n}_{p\sigma} = \hat{a}^\dagger_{p\sigma} \hat{a}_{p\sigma}$ is known as the number operator and K^μ_{pq} is the complex matrix elements forms an anti-Hermitian matrix, whereas $J^\mu_{pq,\sigma\tau}$ is real and it forms a symmetric matrix. To maintain the spin symmetry, $\hat{J}$ should commute with the z-component of the total spin operator $S_z = \frac{1}{2}\sum_p (\hat{n}_{p\alpha} - \hat{n}_{p\beta})$, which is in essence the weighted summation of particle number. Therefore, the conditions like

$$J^\mu_{pq,\alpha\alpha} = J^\mu_{pq,\beta\beta}, \; J^\mu_{pq,\alpha\beta} = J^\mu_{pq,\beta\alpha} \tag{76}$$

should be guaranteed. Meanwhile, note that commutation with S^2 operator requires $J^\mu_{pq,\alpha\alpha} = J^\mu_{pq,\alpha\beta}$, eqn (75) can be reorganized and written as

$$\begin{aligned}\hat{J}_\mu = 2\sum_p J^\mu_{pp,\alpha\beta} \hat{n}_{p\alpha} \hat{n}_{p\beta} + 4\sum_{p<q} J^\mu_{pq,\alpha\beta} \hat{n}_{p\alpha} \hat{n}_{q\beta} + 2\sum_p J^\mu_{pp,\alpha\alpha} [\hat{n}_{p\alpha} + \hat{n}_{p\beta}] \\ + 2\sum_{p<q} J^\mu_{pq,\alpha\alpha} [\hat{n}_{p\alpha} \hat{n}_{q\alpha} + \hat{n}_{p\beta} \hat{n}_{q\beta}]\end{aligned} \tag{77}$$

On the right of the eqn (77), the first term is on-site opposite-spin term; the second one is the non-diagonal opposite-spin term; the third term is the on-site same-spin term, and the last one is the non-diagonal same-spin term.

In eqn (74), the operator $e^{\hat{K}_\mu}$ is also regarded as the rotational operator over the orbitals whose function is to rotate the given basis to new basis. By doing so, the redistribution of the electrons within the orbitals is realized. In this process, the total number of electrons and their spin state are conserved. From the point of view of group theory, the unitary rotational operator $e^{\hat{K}_\mu}$ is a Lie group element while the operator $\hat{K}_\mu$ as its Lie algebra generator. In practice, such a rotation is realized on the quantum computing circuit via a serise of Givens rotation which will be disscussed in details later. In contrast, the number-number interaction operator $\hat{J}_\mu$ realizes the modulation of electron correlation. Therefore, unlike $\hat{K}_\mu$, it is a two-body operator. The sandwitch shape structure $e^{\hat{K}_\mu}e^{i\hat{J}_\mu}e^{-\hat{K}_\mu}$ composed of those two types of operators is a similarity transformatioFn. When it is applied onto a reference state like we saw in eqn (74), it implies three steps. First, it rotates reference state $|\psi_0\rangle$ into new orbital basis via $e^{-\hat{K}_\mu}$, then $e^{i\hat{J}_\mu}$ will exert electron correlation modulation on the new basis, i.e. the Jastrow factor. After that, it will change the basis back via $e^{\hat{K}_\mu}$.

Now, suppose we are going to use the widely used JW mapping scheme (see section 2.4), N spatial orbitals should be mapped onto a system of $2N$ qubits,

$$\hat{a}^\dagger_{p\alpha} \mapsto (S_+)_p Z_{(p-1)}...Z_0,\ \hat{a}^\dagger_{p\beta} \mapsto (S_+)_{N+p} Z_{N+p-1}...Z_0, \tag{78}$$

where $(S_+)_p = (X_p + iY_p)/2$ is the raising operator for qubit p. From here we can see, to achieve linear connectivity, we need $\mathcal{O}(N^2)$ Givens rotations acting on adjacent qubits. Considering eqn (77), the operator $e^{i\hat{J}_\mu}$ is mapped onto a quantum circuit of the form

$$\begin{aligned} e^{i\hat{J}_\mu} \mapsto &\prod_p v_{p,p+N}(2J^\mu_{pp,\alpha\beta}) \prod_{p<q} v_{p,q+N}(4J^\mu_{pq,\alpha\beta}) \prod_p w_p(2J^\mu_{pp,\alpha\alpha}) w_{p+N}(2J^\mu_{pp,\alpha\alpha}) \\ &\times \prod_{p<q}(2J^\mu_{pq,\alpha\beta}) v_{p+N,q+N}(2J^\mu_{pq,\alpha\beta}), \end{aligned} \tag{79}$$

where $w_p(\phi) = e^{-i\frac{\phi}{4}Z_p}$ is a single-qubit gate and

$$v_{p,q}(\phi) = e^{-i\frac{\phi}{4}(Z_p + Z_q - Z_pZ_q)} \tag{80}$$

is a two-qubit gate. Eqn (79) indicates that we need $\mathcal{O}(N^2)$ two-qubit gates with all-to-all connectivity. In summary, we can say the UCJ is not applicabale to current hardware.

5.6.2 Local Unitary Cluster Jastrow (LUCJ)

To overcome the problem of UCJ addressed in the previous subsection, we introduce "local" approximation of the UCJ ansatz, which needs the following modifications for opposite-spin

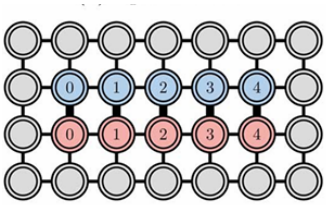

图 8: Square Connectivity

and same-spin number-number terms:

$$\sum_{pq} J_{p\alpha,q\beta}\hat{n}_{p\alpha}\hat{n}_{p\beta} \rightarrow \sum_{p} J_{p\alpha,p\beta}\hat{n}_{p\alpha}\hat{n}_{p\beta} \tag{81}$$

$$\sum_{pq} J_{p\sigma,q\sigma}\hat{n}_{p\sigma}\hat{n}_{q\sigma} \rightarrow \sum_{p} J_{p\sigma,(p+1)\sigma}\hat{n}_{p\sigma}\hat{n}_{(p+1)\sigma}, \text{ where } \sigma = \alpha, \beta \tag{82}$$

From the two equations above, we can see the summation over p and q is simplified into summation of p only. By doing so, we neglect all the terms with $p \neq q$ so that only diagonal double occupation penalty terms are kept for opposite-spin terms. As for the same-spin term, the summation is performed over neigboring terms. Obviously, this approximation only takes into account the strongest electronic repulsion. In other words, two sparse matrices with the elements of $J^{\mu}_{p\alpha,q\alpha}$ and $J^{\mu}_{p\alpha,q\beta}$ respectively are formed. This allows us to implement the circuit in a constant depth with limited connectivity. For example, if the connectivity is a square lattice show in the figure 8. In vector form, the summation corresponding to eqns (81, 82)

$$\begin{aligned} \boldsymbol{J}^{\alpha\alpha} &: \{(p, p+1), p = 0, ..., N-2\} \\ \boldsymbol{J}^{\alpha\beta} &: \{(p, p), p = 0, ..., N-1\} \end{aligned} \tag{83}$$

Similarly, in the case of the heavy-hex lattice (see fig 9), the sparse matrices are

$$\begin{aligned} \boldsymbol{J}^{\alpha\alpha} &: \{(p, p+1), p = 0, ..., N-2\} \\ \boldsymbol{J}^{\alpha\beta} &: \{(p, p), p = 0, 4, 8, ...(p \leq N-1)\} \end{aligned} \tag{84}$$

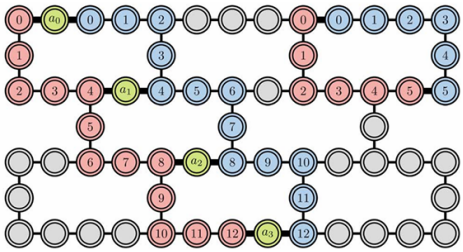

图 9: Heavy-hex lattice

6 Paper-reading related to adaptive grid method

In the paper *Real-Space Chemistry on Quantum Computers: A Fault-Tolerant Algorithm with Adaptive Grids and Transcorrelated Extension* [3], an adaptive grid method for quantum chemistry presented in the form of first-quantized real-space formulation is given. The following subsections are about my investigation of this paper and the collection of my understanding related to the topics contained in this paper.

6.1 Voronoi Diagrams and Discretization of Wavefunction

This subsection is dedicated the understanding of Eqn. (13) in this paper, which is about the discretization of a single electron wavefunction onto a multi-center grid with N points, and it looks like

$$|\psi_e\rangle = \sum_{m=0}^{N-1} c_m |m\rangle , \tag{85}$$

where m runs through the indices of the cells in the Voronoi diagram.

To understand this formula correctly, the word "discretization" is the key. Using Voronoi diagram, we have divided the 3D space into N non-overlapping cells. Each cell corresponds to a grid point. Since $|m\rangle$ is a discretized base, it means although the base is defined on the whole 3D space, it concentrates on the corresponding mth Voronoi cell with complex amplitude c_m which fulfils $\sum_{m=0}^{N-1} |c_m|^2 = 1$. That means on the Voronoi cell $Vor(m)$, value of $|m\rangle$ is 1, and zero anywhere else. Therefore, unlike the continuous wavefunction, we do not need to consider the continuous boundary condition between each Voronoi cells. In the Hilbert space, the basis $|m\rangle$ forms an orthonormal basis which fulfils

$$\langle m'| m\rangle = \delta_{m'm}. \tag{86}$$

Example 6.1. *Suppose we have divided the whole space into three Voronoi cells, corresponding to three basis:* $|0\rangle = (1,0,0), |1\rangle = (0,1,0)$ *and* $|2\rangle = (0,0,1)$*. Then the wavefunction can be written into*

$$|\psi\rangle = c_0 |0\rangle + c_1 |1\rangle + c_2 |2\rangle = (c_0, c_1, c_2). \tag{87}$$

The scenario discussed above is for a single electron. If we have two electrons, each of them has 3 cells, then the dimensionality of the whole Hilbet space is $3 \times 3 = 9$*, and the wavefunction can be expressed as*

$$|\Psi\rangle = \sum_{n,n=0}^{2} c_{mn} |m\rangle_1 \otimes |n\rangle_2 , \tag{88}$$

which is mentioned later in the paper the tensor product of η *single-electron wavefunctions that lives in* $\mathcal{H}_{tot} = \bigotimes_{i=1}^{\eta} \mathcal{H}_e$*.*

6.2 Pauli decomposition

6.2.1 Equation 22

In order to understand the equation (22) in this paper, which looks like

$$\bar{\mathbf{H}} = \sum_{n,n=0}^{N-1} \omega_{mn} \sum_{i=0}^{\eta-1} \prod_{b=0}^{\log N-1} \mathbf{X}_{i\log N+b}^{m_b} \mathbf{Z}_{i\log N+b}^{n_b} + \frac{1}{2} \sum_{m,p=0}^{N-1} \gamma_{mp} \sum_{i\neq j}^{\eta-1} \prod_{b=0}^{\log N-1} \mathbf{Z}_{i\log N+b}^{m_b} \mathbf{Z}_{j\log N+b}^{p_b}, \quad (89)$$

we track down to the paper *Pauli Decomposition via the Fast Walsh-Hadamard Transform*[4], which is a paper descibing the Pauli Linear Combination of Unitaries (LCU) decomposition.

Within the paper [4], the collection of the Pauli matrices (see Table 4), are included in a set $\mathscr{P} \coloneqq \{I, X, Y, Z\}$. Based on that, a set $\mathscr{P}_n$ of the dimension $\mathbb{C}^{2n\times 2n}$ is defined by

$$\mathscr{P}_n \coloneqq \left\{ \bigotimes_{j=0}^{n-1} P_j \ : \ P_j \in \mathscr{P} \right\} \quad (90)$$

where n is an integer. Then any complex matrix $A \in \mathbb{C}^{2n\times 2n}$ can be expanded into a weighted sum of tensor products as

$$A = \sum_{P\in\mathscr{P}} \alpha(P) P, \quad (91)$$

where the mapping $\alpha \coloneqq \mathscr{P}_n \to \mathbb{C}$ is called the Pauli coefficient which is theoretically given by

$$\alpha(P) = \frac{1}{2^n} \mathrm{tr}(A^\dagger P) \text{ where } P \in \mathscr{P}_n. \quad (92)$$

The problem to be addressed here is to compute the Pauli coefficients efficiently, since direct computation over Eqn (92) has the scaling of $\mathcal{O}(2^{5n})$. Plus, this equation is NOT easy to be used for LCU of a Hamiltonian or a matrix used in specific applications.

To overcome the difficulty, the authors of paper [4] proved the following theorem first. This theorem is expressed as:

Theorem 6.1. *(Pauli decomposition of $A \in \mathbb{C}^{2n\times 2n}$ via the Walsh-Hadamard transfrom). The following decomposition*

$$A = \sum_{r,s=0}^{2^n-1} \alpha_{r,s} P_{r,s}, \quad (93)$$

where

$$P_{r,s} \coloneqq \bigotimes_{j=0}^{n-1} (i^{r_j \wedge s_j} X^{r_j} Z^{s_j}) \in \mathscr{P}_n \quad (r, s \in [2^n - 1]_0) \quad (94)$$

and

$$\alpha_{r,s} \coloneqq \frac{i^{-|r\wedge s|}}{2^n} \sum_{q=0}^{2^n-1} a_{q\oplus r,q} (H^{\otimes n})_{q,s} \quad (r, s \in [2^n - 1]_0) \quad (95)$$

holds. Conversely, "A" can be recovered from its Pauli string decomposition according to

$$\alpha_{r,s} = i^{|\bar{r}\wedge s|} \sum_{q=0}^{2^n-1} \alpha_{r\oplus s,q} (H^{\oplus n})_{q,s}. \tag{96}$$

In Theorem(6.1), the symboles $\wedge, \oplus$ and $\bar{\cdot}$ are the bitwise AND, XOR, and NOT operators. The symbole $|\cdot|$ denotes the Hamming weight. For a given integer N, we define $[N]_0 \coloneqq \{0, 1, \ldots, N\}$. Therefore,

$$[2^n - 1]_0 \coloneqq \{0, 1, \ldots, 2^n - 1\}. \tag{97}$$

r_j and s_j are the binary expansion coefficients of r and s, respectively. For the purpose of convenient reference, the well-known binary expansion for an integer p is given as

$$p = \sum_{j=0}^{n-1} p_j 2^j \quad (p_j \in \{0, 1\}) \tag{98}$$

which defines a map $p \mapsto (p_j)_{j=0}^{n-1} \in \{0, 1\}^n$. The bitwise XOR and AND are defined according to

$$\mathrm{XOR} : p \oplus q \coloneqq \sum_{j=0}^{n-1} (p_j \oplus q_j) 2^j, \quad \mathrm{AND} : p \wedge q \coloneqq \sum_{j=0}^{n-1} (p_j \wedge q_j) 2^j \quad (p, q \in [2^n - 1]_0). \tag{99}$$

The Hamming weight is calculated according to

$$|p| \coloneqq \sum_{j=0}^{n-1} p_j \quad (p \in [2^n - 1]_0). \tag{100}$$

Example 6.2. *Suppose we are studying a system of 3 qubits, which means $n = 3$. According to Eqn. (97), the number set we are dealing with is $\{0, 1, \ldots, 7\}$. Let $p = 6$ and $q = 7$, we can easily see their binary forms are $p = 6_{10} = 110_2$ and $q = 7_{10} = 111_2$, respectively. Now, by making use of Eqn. (99), we can find the results of bitwise operator between p and q. For XOR, it is $p \oplus q = 6 \oplus 7 = (0 \oplus 1)2^0 + (1 \oplus 1)2^1 + (1 \oplus 1)2^2 = 1$. For AND, it is $p \wedge q = 6 \wedge 7 = (0 \wedge 1)2^0 + (1 \wedge 1)2^1 + (1 \wedge 1)2^2 = 6$. Of course, for Hamming weight, for $p = 6$, it is 0+1+1=2*

Define a matrix unit for $\mathbb{C}^{2n\times 2n}$ $E_{p,q}$ as

$$E_{p,q} \coloneqq (\delta_{j,p}\delta_{k,q})_{j,k=0}^{2^n-1} \quad (p, q \in [2^n - 1]_0), \tag{101}$$

which can also be written in the form

$$E_{p,q} = \bigotimes_{j=0}^{n-1} e_j(p, q), \tag{102}$$

where

$$e_j(p,q) \coloneqq (\delta_{k,p_j}\delta_{l,q_j})_{k,l=0}^{1} \quad (j \in [n-1]_0; p,q \in [2^n-1]_0). \tag{103}$$

Once again, those three equations can be checked by using a special scenario such that $n = 3, p = 6$ and $q = 7$. According to Eqn (103), we will obtain an 8*8 matrix in which only the element in row 7 and column 8 is 1 and the rest elements are all zeros. This 8*8 matrix can be expanded into $\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \otimes \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \otimes \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, which confirms the Eqn. (102) and Eqn. (103). Using the Pauli matrices as the basis, Eqn. (103) can be written into

$$e_j(p,q) = \frac{1}{2}(X^{p_j \oplus q_j} + (-1)^{p_j}(iY)^{p_j \oplus q_j} Z^{\overline{p_j \oplus q_j}}). \tag{104}$$

Considering $XZ = -iY$, Eqn. (102) can be modified into

$$\begin{aligned} E_{p,q} &= \frac{1}{2^n} \bigotimes_{j=0}^{n-1} (X^{p_j \oplus q_j} + (-1)^{q_j} X^{p_j \oplus q_j} Z) \\ &= \frac{1}{2^n} \bigotimes_{j=0}^{n-1} \sum_{s_j=0,1} (-1)^{q_j \wedge s_j} X^{p_j \oplus q_j} Z^{s_j} \\ &= \frac{1}{2^n} \prod_{j=0}^{n-1} \sum_{s_j=0,1} (-1)^{q_j \wedge s_j} X_j^{p_j \oplus q_j} Z_j^{s_j}, \end{aligned} \tag{105}$$

with

$$X_j \coloneqq I^{\otimes j} \otimes X \otimes I^{\otimes(n-j-1)}, \quad Z_j \coloneqq I^{\otimes j} \otimes Z \otimes I^{\otimes(n-j-1)} \quad (j \in [n-1]_0). \tag{106}$$

The last step of Eqn. (105) makes sense because the operator $(-1)^{q_j \wedge s_j} X^{p_j \oplus q_j} Z^{s_j}$ only exert on the j^{th} qubit, and with the definition from Eqn. (106) the last expression in Eqn. (105) will do the same job. Switching the sequence of the summation and production, the last expression can be written as

$$E_{p,q} = \frac{1}{2^n} \left(\sum_{s_0=0,1} \cdots \sum_{s_{n-1}=0,1} \right) \prod_{j=0}^{n-1} (-1)^{q_j \wedge s_j} X^{p_j \oplus q_j} Z^{s_j} \tag{107}$$

By transforming the $E_{p,q}$ into the form above, we can recognize immediately the series summation within the round brackets are actually the summation over all the possible combination of binary number generated from the integer n. Use $n = 3$ as an example again, those summations will run through 000_2 all the way to 111_2, which means all the 8 possible combinations. Therefore, all the binary summation can be replaced by summation over integers, which is

$$E_{p,q} = \frac{1}{2^n} \sum_{s=0}^{2^n-1} \prod_{j=0}^{n-1} (-1)^{q_j \wedge s_j} X^{p_j \oplus q_j} Z^{s_j}. \tag{108}$$

Notice that the tensor product of Hadamard matrices

$$H_{q_j,s_j} = (-1)^{q_j \wedge s_j} \quad (q_j, s_j \in \{0,1\}) \tag{109}$$

is

$$(H^{\otimes n})_{q,s} = \prod_{j=0}^{n-1} (-1)^{q_j \wedge s_j} \quad (q, s \in [2^n - 1]_0), \tag{110}$$

Eqn. (108) can be further modified into

$$E_{p,q} = \frac{1}{2^n} \sum_{s=0}^{2^n-1} (H^{\otimes n})_{q,s} \prod_{j=0}^{n-1} X^{p_j \oplus q_j} Z^{s_j}. \tag{111}$$

With the help of the equation above, the decomposition of A into a sum of elementary matrices can be expressed as

$$A = \frac{1}{2^n} \sum_{p,q,s=0}^{2^n-1} a_{p,q} (H^{\otimes n})_{q,s} \prod_{j=0}^{n-1} X^{p_j \oplus q_j} Z^{s_j}. \tag{112}$$

By introducing a new summation index $r \coloneqq p \oplus q$, Eqn. (112) can be rewritten as

$$\begin{aligned} A &= \frac{1}{2^n} \sum_{r,s=0}^{2^n-1} \sum_{q=0}^{2^n-1} a_{q \oplus r, q} (H^{\otimes n})_{q,s} \prod_{j=0}^{n-1} X^{p_j \oplus q_j} Z^{s_j} \\ &= \sum_{r,s=0}^{2^n-1} \left(\frac{i^{-|r \wedge s|}}{2^n} \sum_{q=0}^{2^n-1} a_{q \oplus r, q} (H^{\otimes n})_{q,s} \right) \left(i^{|r \wedge s|} \prod_{j=0}^{n-1} X^{p_j \oplus q_j} Z^{s_j} \right) \end{aligned} \tag{113}$$

6.2.2 Equation 24

Now, let's turn our attention to Eqn. (24) in [3], which looks like

$$\omega_{mn} = \frac{1}{N} \sum_{x=0}^{N-1} (-1)^{x \odot n} \bar{\mathbf{T}}_{m \oplus x, x}. \tag{114}$$

In the following paragraphs, we will prove this expression is a Walsh-Hadamard transformation between ω_{mn} and $\bar{\mathbf{T}}_{xy}$, which can be written in the matrix form as:

$$\omega = \mathbb{H} \bar{\mathrm{T}} \mathbb{H}, \tag{115}$$

where $\mathbb{H}$ denotes Hadamard matrix. We start from the simplest 1D Walsh-Hadamard transformation by defindingg a function f on $\{0,1\}^k$ such that

$$f(x) : \{0,1\}^k \to C, \quad x = (x_0, ..., x_{k-1}). \tag{116}$$

And the *bitwise dot product* is defined as

$$x \odot n \equiv \sum_{b=0}^{k-1} x_b n_b \,(\mathrm{mod}\, 2), \tag{117}$$

which contains three procedures: **1**. multiplying every binary digits $x_b n_b$; **2**. summing up the products; **3**. performing mod 2, keeping its parity as the final result. So, the result of Eqn. (117) is always binary. Such a binary result reflects the phase of Pauli Z.

Based on the preceding definitions, the Walsh-Hadamard transformation (WHT) is defined as

$$\hat{f}(n) = \frac{1}{2^{k/2}} \sum_{x \in \{0,1\}^k} (-1)^{x \odot n} f(x). \tag{118}$$

Its corresponding matrix form is

$$\hat{f} = \mathbb{H} f, \tag{119}$$

where $\mathbb{H} \in C^{2^k \times 2^k}$ is the normalized Hadamard matrix, whose (n, x) entry is

$$\mathbb{H}_{nx} = \frac{1}{2^{k/2}} (-1)^{x \odot n}. \tag{120}$$

From Eqn. (118), we should recongnize WHT is in essence the Fourier transformation on the Boolean domain $\{0,1\}^k$, with the new transformation kernel $(-1)^{x \odot n}$ rather than the regular kernel like e^{-ikx} in the complex domain. For this reason, it will be convenient for us to define *Hadamard convolution* (XOR convolution) on the Boolean domain as

$$(f *_{\oplus} g)(x) = \sum_y f(y) g(x \oplus y), \tag{121}$$

which is parallel of the regular convolution $(f * g)(x) = \sum_y f(y) g(x - y)$ in the regular space. From Eqn. (121), after performing WHT on it, we obtain

$$\widehat{f *_{\oplus} g}(n) = \hat{f}(n) \hat{g}(n) \tag{122}$$

Now, let's consider a matrix

$$\bar{\mathbf{T}} : \{0,1\}^k \times \{0,1\}^k \to C, \quad \bar{\mathbf{T}}_{ab}, \tag{123}$$

where $a, b \in \{0,1\}^k$ are the binary indices. By now we can see Eqn. (115) is actually a two-sided WHT on $\bar{\mathbf{T}}$. Substituting Eqn. (120) into Eqn. (115), we can figure out its components form, which is

$$\begin{aligned} \omega_{mn} &= \sum_{a,b \in \{0,1\}^k} \mathbb{H}_{ma} \bar{\mathbf{T}}_{ab} \mathbb{H}_{bn} \\ &= \frac{1}{2^k} \sum_{a,b} (-1)^{a \odot m} \bar{\mathbf{T}}_{ab} (-1)^{b \odot n}. \end{aligned} \tag{124}$$

Let $x \coloneqq b$ and $a \coloneqq m \oplus x$, since $(a, b) \mapsto (m \oplus x, x)$ is a bijection on $\{0,1\}^k$, the summation in Eqn. (124) can be rewritten as

$$\omega_{mn} = \frac{1}{2^k} \sum_{x \in \{0,1\}^k} (-1)^{(m \oplus x) \odot m} \bar{\mathbf{T}}_{m \oplus x, x} (-1)^{x \odot n}. \tag{125}$$

Considering $(m \oplus x) \odot m = m \odot m + x \odot m \equiv x \odot m \pmod 2$ and $m \odot m$ is a constant independent of x, which will only lead to a fixed phase $(-1)^{m \odot m}$ that can be absorbed into $\mathbb{H}$, the equation above is equivalent to Eqn. (114).

6.3 Quantum Chebyshev Phase Estimation

Since the transcorrelated Hamiltonian matrix is non-Hermitian, it is no longer feasible to find its eigenvalues with a quantum algorithm by usual Quantum Phase Estimation (QPE). Therefore, a new method called Qunatum EigenValue Estimation (QEVE) [14] is needed. In this algorithm, given a v qubit register, a state $|\psi\rangle \in \mathbb{C}^{N^\eta}$ that is prepared close to the ground state of $\tilde{\mathbf{H}}$. Becasue no one can guarantee that $\tilde{\mathbf{H}}$ is unitary, which means we are not sure whether it can be handled by qubits, we embed the $\tilde{\mathbf{H}}$ into a larger unitary matrix U like

$$U = \begin{bmatrix} \frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}} & \cdot \\ \cdot & * \end{bmatrix}, \tag{126}$$

where $\alpha_{\tilde{\mathbf{H}}}$ is called subnormalization constant, which is defined by $\alpha_{\tilde{\mathbf{H}}} \geq \|\tilde{H}\|_2$, indicating the maximum of the L2 norm of $\tilde{\mathbf{H}}$. To convince ourselves that we can always construct a unitary matrix out of random given matrix A, we can build up a matrix like

$$U' = \begin{bmatrix} A/\alpha & \sqrt{I - A^\dagger A/\alpha^2} \\ \sqrt{I - A^\dagger A/\alpha^2} & -A^\dagger/\alpha \end{bmatrix}. \tag{127}$$

It is easy to verify that the U' is unitary no matter whether A is Hermitian or reversible or sparse, or its shape.

Now, suppose we have a ancilla register, from the requirement

$$\langle 0^a | \, U \, | 0^a \rangle = \frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}}, \tag{128}$$

we can see the information contained in $\tilde{\mathbf{H}}$ is now embedded in the bigger unitary matrix U, and this information can be extracted. So, in terms of information, U and $\tilde{\mathbf{H}}$ are equivalent. For our convenience, in the following discussion we use $\tilde{\mathbf{H}}$ only, but keep in mind, if you see the operator like $T_k(\frac{\tilde{\mathbf{H}}}{\alpha})$, it is realized via U.

Since we are now having a unitary matrix (operator) to play with, we can construct a state like

$$\sum_{\ell=0}^{v-1} |\ell\rangle\, e^{2\pi i \ell \frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}}} |\psi\rangle\,. \tag{129}$$

In this equation, $|\psi\rangle$, which has already been meantioned before, is constructed by an oracle formed by the v register, via $O_\psi |0\rangle = |\psi\rangle$. Hopefully, the state we construced via O_ψ is close enough to the real eigenstate $|\psi_0\rangle$ of the system. Ideally, when $|\psi\rangle = |\psi_0\rangle$, it is easy to see, the eigenvalue, which is our ultimate aim, is now embedded in the phase of the exponential function on the left of $|\psi\rangle$ in Eqn. 129. This idea is called phase kick-back, and it can be described mathematically, on the condition that $\tilde{H}|\psi\rangle = \lambda|\psi\rangle$, as

$$\sum_{\ell=0}^{v-1} |\ell\rangle\, e^{2\pi i \ell \frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}}} |\psi\rangle = \sum_{\ell=0}^{v-1} |\ell\rangle\, e^{2\pi i \ell \frac{\lambda}{\alpha_{\tilde{H}}}} |\psi\rangle = \sum_{\ell=0}^{v-1} e^{2\pi i \ell \frac{\lambda}{\alpha_{\tilde{H}}}} |\ell\rangle \otimes |\psi\rangle\,. \tag{130}$$

The λ contained in the phase can be measured via a QFT on the O_ψ.

However, as indicated by [20], if we define $\epsilon \in (0,1)$ as the error tolerance, algorithm based on Eqn. 129 suffers from a suboptimal $\mathcal{O}(\epsilon^{-1} \cdot \mathrm{polylog}(\epsilon^{-1}))$ query complexity. To address this problem, an alternative QEVE based on *Chebyshev history states* was proposed, obtaining the $\mathcal{O}(1/\epsilon)$ of QPE. To understand the *Quantum Chebyshev Phase Estimation* (QCPE) algorithm, we need to explain what the Chebyshev polynomialis and its relevant properties. By doing so, we will know why we apply this particular polynomial in QCPE.

6.3.1 Chebyshev Polynomials of the First and Second Kinds

The ℓth Chebyshev polynomial of the first kind $T_\ell : [-1;1] \to [-1;1]$ is defined as:

$$T_\ell(x) = \cos(\ell \arccos(x)). \tag{131}$$

This trignometrical structure implies its compatibility of QFT, that is one of the reasons Chebyshev polynomial is chosen. Meanwhile, we notice that Chebyshev polynomial is a mapping from interval [-1;1] to [-1;1], that is reason when we constructed the block encoding (see Eqn. (126)), $\tilde{H}$ is divided by the maximum norm $\alpha_{\tilde{H}}$. Later, to generate the *Chebyshev history state*, the following generating function

$$\sum_{\ell=0}^{\infty} T_\ell(x) y^\ell = \frac{1 - yx}{1 + y^2 - 2yx} \tag{132}$$

is used [14] after it is rescaled into

$$\tilde{T}_\ell(x) = \begin{cases} \frac{1}{2} T_0(x) & \text{if} \quad \ell = 0 \\ T_\ell(x) & \text{if} \quad \ell > 0 \end{cases}, \tag{133}$$

with new generating function

$$\sum_{\ell=0}^{\infty} \tilde{T}_\ell y^\ell = \frac{1-y^2}{2(1+y^2-2yx)}. \tag{134}$$

Besides the first kind Chebyshev polynomial, we also have the second form, which is defined as:

$$U_\ell(x) = \frac{\sin((\ell+1)\arccos(x))}{\sin\arccos(x)} \tag{135}$$

with generating function

$$\sum_{\ell=0}^{\infty} U_\ell(x) y^\ell = \frac{1}{1+y^2-2yx}. \tag{136}$$

For more detailed description, please refer to Appendix B.1.

6.3.2 Chebyshev History State

With the definition of Chebyshev polynomial, we use

$$\sum_{\ell=0}^{v-1} |\ell\rangle \otimes T_\ell\left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}}\right)|\psi\rangle \tag{137}$$

to take the place of Eqn. 129. The total systems are said to form a *Chebyshev history state* of $\tilde{\mathbf{H}}$ once Eqn. 137 is fulfilled. The reason we call it *history state* is that the v qubits register $|\ell\rangle$ serves as a counter that indicates the order of the Chebyshev polynomial of $\tilde{\mathbf{H}}/\alpha_{\tilde{H}}$ applied on the state $|\psi\rangle$ in the sum. Follow the same idea we obtain Eqn. (130), we find the phase $\phi = \frac{1}{2\pi}\arccos\frac{\lambda}{\alpha_{\tilde{H}}}$ is kicked back and end up with

$$\sum_{\ell=0}^{v-1} \cos(2\pi\ell\phi)\,|\ell\rangle \otimes |\psi\rangle\,. \tag{138}$$

If we only concentrate on the cosine part together with $|\ell\rangle$ for now, after performing the QFT, we get

$$\mathrm{QFT}\sum_{\ell=0}^{v-1} \cos(2\pi\ell\phi)\,|\ell\rangle = \frac{1}{2\sqrt{v}}\sum_{\ell=0}^{v-1}\sum_{\ell'=0}^{v-1}\left(e^{2\pi i\ell'(\phi-\frac{\ell}{v})} + e^{-2\pi i\ell'(\phi+\frac{\ell}{v})}\right). \tag{139}$$

According to [14], on the condition that the scale $\alpha_{\tilde{H}} > 2\|\tilde{\mathbf{H}}\|$ is applied, by measuring ℓ in the computational basis, we obtain an approximation of $v\phi$ with a success probability greater than $1/2$, which means the desired quantum state dominates, and we can boost its probability to near 1 with repeated applications. The success probability can then be boosted to at least $1-p_f$ by repeating the procedure $\mathcal{O}(\log(1/p_f))$ times.

In order to introduce the matrix form of Chebyshev history state, let us define and study a $v \times v$ *lower shift operator*

$$L = \sum_{k=0}^{n-2} |k+1\rangle \langle k| \,. \tag{140}$$

From the following example, we can see why we call it lower shift operator and one important property.

Example 6.3. *Let us construct L_4, a 4×4 lower shift matrix:*

$$\begin{aligned} L_4 &= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix} \end{aligned} \tag{141}$$

and we also have

$$L_4 |2\rangle = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |3\rangle \,, \tag{142}$$

as well as

$$L_4 |3\rangle = \vec{0}. \tag{143}$$

*Eqn. (142) shows us that $L_v |k+1\rangle = |k\rangle$, hence the name **lower shift**. Eqn. (143) tells us there is a truncation in the series of operations.*

With the help of the lower shift operator $\mathbf{L}_v^\ell = \sum_{\ell'=0}^{v-1-\ell} |\ell + \ell'\rangle \langle \ell'|$, we can construct the matrix version of the operator of the generating function for rescaled Chebyshev polynomials

$$G\left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}}\right) = \sum_{\ell=0}^{v-1} \mathbf{L}_v^\ell \otimes \tilde{T}_\ell \left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}}\right) \tag{144}$$

applied on the $|0\rangle \otimes |\psi\rangle$ state gives the rescaled Chebyshev history state

$$\sum_{\ell=1}^{v-1} |\ell\rangle \otimes \tilde{T}_\ell \left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}}\right) |\psi\rangle \,. \tag{145}$$

If Eqn. (144) is applied onto the zeroed v qubit register, we can see its complete Chebyshev history state:

$$\sum_{\ell=0}^{v-1} \mathbf{L}_v^\ell \otimes \tilde{T}_\ell \left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}} \right) = \sum_{\ell=0}^{\infty} \mathbf{L}_v^\ell \otimes \tilde{T}_\ell \left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}} \right) = \frac{\mathbf{I}_v \otimes \mathbf{I} - \mathbf{L}_v^2 \otimes \mathbf{I}}{2Pad\left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}} \right)}, \tag{146}$$

where $Pad\left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}} \right)$ is defined as

$$\begin{aligned} Pad\left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}} \right) &= \mathbf{I}_v \otimes \mathbf{I} + \mathbf{L}_v^2 \otimes \mathbf{I} - 2\mathbf{L} \otimes \frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}} \\ &= \begin{bmatrix} \mathbf{I} & 0 & \cdots & \cdots & \cdots & 0 \\ -2\tilde{\mathbf{H}}/\alpha_{\tilde{H}} & \mathbf{I} & \ddots & \ddots & \ddots & \vdots \\ \mathbf{I} & -2\tilde{\mathbf{H}}/\alpha_{\tilde{H}} & \mathbf{I} & \ddots & \ddots & \vdots \\ 0 & \mathbf{I} & -2\tilde{\mathbf{H}}/\alpha_{\tilde{H}} & \mathbf{I} & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & \mathbf{I} & -2\tilde{\mathbf{H}}/\alpha_{\tilde{H}} & \mathbf{I} \end{bmatrix} \end{aligned} \tag{147}$$

6.3.3 Brief summary of this algorithm

The following content is from [3].

1. Construct a block encoding of the target matrix $\tilde{\mathbf{H}}$ and the lower shift matrix $\tilde{\mathbf{L}}_v$.
2. Use $\mathbf{L}_v$ and $\tilde{\mathbf{H}}$ to form the $Pad\left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}} \right)$ gate according to Eqn. (147).
3. Prepare the v qubit register in the state $\frac{|0\rangle - |2\rangle}{2}$.
4. Invoke the quantum linear system solver the equation

$$Pad\left(\frac{\tilde{\mathbf{H}}}{\alpha_{\tilde{H}}} \right) |\Phi\rangle \otimes |\psi\rangle = \frac{|0\rangle - |2\rangle}{2} \otimes |\psi\rangle \tag{148}$$

5. Perform the QFT and measure in the computational basis.
6. Boost the success probability using median amplification.

6.4 Transcorrelated Molecular Hamiltonian

This subsection is dedicated to the interpretation of the Eqns. (31, 32, 33) in [3], which look like

$$\begin{aligned}\tilde{H} = \hat{H} - \sum_{i,\alpha} \tilde{\mathcal{K}}[g_{\mu_{ne}}](\mathbf{x}_i, \mathbf{R}_\alpha) - \sum_{i,\alpha<\beta} \tilde{\mathcal{B}}[g_{\mu_{ne}}(\mathbf{x}_i, \mathbf{R}_\alpha, \mathbf{R}_\beta)] \\ - \sum_{i<j} \tilde{\mathcal{K}}[h_{\mu_{ee}}](\mathbf{x}_i, \mathbf{x}_j) - \sum_{i<j<k} \tilde{\mathcal{B}}[h_{\mu_{ee}}](\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k)\end{aligned} \tag{149}$$

with

$$\begin{aligned}\tilde{\mathcal{K}}[h_{\mu_{ee}}](\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{2}\left[\nabla_i^2 h + \nabla_j^2 h + (\nabla_i h)^2 + (\nabla_j h)^2\right] \\ + \nabla_i h \cdot \nabla_i + \nabla_j h \cdot \nabla_j\end{aligned} \tag{150}$$

and

$$\begin{aligned}\tilde{\mathcal{B}}[h_{\mu_{ee}}](\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) = \nabla_i h_{ij} \cdot \nabla_i h_{ik} + \nabla_j h_{ji} \cdot \nabla_j h_{jk} \\ + \nabla_k h_{ki} \cdot \nabla_k h_{kj},\end{aligned} \tag{151}$$

where $h \equiv h_{\mu_{ee}}$, $h_{ij} = h(\mathbf{x}_i, \mathbf{x}_j)$ and $\nabla_i \equiv \nabla_{\mathbf{x}_i}$.

In Eqn. (149), the $\tilde{H}$ is called the transcorrelated (TC) Hamiltonian of $\hat{H}$. Suppose $\hat{H}$ has an eigenvalue equation $\hat{H}\Psi = E\Psi$. Now, let

$$\Psi = e^{\tau}\Phi, \tag{152}$$

The eigenvalue equation is turned into $\hat{H}e^{\tau}\Phi = Ee^{\tau}\Phi$ and can be further transformed into $e^{-\tau}\hat{H}e^{\tau} = E\Phi$, where we can find the definition of $\tilde{H}$ and the fact that $\tilde{H}$ has the same spectrum of $\hat{H}$. Based on the Eqn. (29), which is Eqn. (154) in this note, the Hamiltonian

$$\hat{H} = \sum_{i=0}^{\eta-1}\left(-\frac{1}{2}\nabla_{\mathbf{x}_i}^2 - \sum_{\alpha=0}^{M-1}\frac{\mathcal{Z}_\alpha}{|\mathbf{x}_i - \mathbf{R}_\alpha|} + \sum_{j>i}^{\eta-1}\frac{1}{|\mathbf{x}_i - \mathbf{x}_j|}\right) \tag{153}$$

with a symmetrical function of the η-electron coordinates defined as

$$\tau(\mathbf{x}_i) = \sum_{i=0}^{\eta-1}\left(\sum_{\alpha=0}^{M-1} g_{\mu_{ne}}(x_{i\alpha}) + \sum_{j>i}^{\eta-1} h_{\mu_{ee}}(x_{ij})\right) \tag{154}$$

will be converted into TC Hamiltonian so that the TC eigenfunctions are free of cusps. In Eqn. (154), $g_{\mu_{ne}}$ and $h_{\mu_{ee}}$ are functions of the electron-nuclei and electron-electron distances $x_{i\alpha} = |\mathbf{x}_i - \mathbf{R}_\alpha|$ and $x_{ij} = |\mathbf{x}_i - \mathbf{x}_j|$. *Comments: Although Eqn. (154) is a direct copy of Eqn. (30) from [3], I don't think it is a well written expression. The reason is on the right side of this equation, the dummy variable "i" has been summed, so it should not appear as a footnote on the left side of equation.*

6.4.1 My wasted effort

With Eqn. (153, 154), we can now analyze $[\hat{H}, \tau]$. We immediately notice from Eqn. (153) that $\hat{H}$ is composed of the kinematic energy term which is a second-order derivative, and the potential terms which are only functions. Therefore,

$$[\hat{H}, \tau] \to -\frac{1}{2}\nabla^2_{\mathbf{x}_i}\tau + \tau\frac{1}{2}\nabla^2_{\mathbf{x}_i}, \tag{155}$$

since scalar functions are commutative with each other. To figure out the $\nabla^2_{\mathbf{x}_i}\tau$, let's temporarily study a simplified system of two nucleis and three electrons so that we can get rid of the complexity caused by the nested summation. The three electrons can be labelled by 0, 1 and 2, and the two nucleis can be labelled by α and β, respectively. Based on these assumptions, we have

$$\begin{aligned}
&(\nabla^2_{\mathbf{x}_0} + \nabla^2_{\mathbf{x}_1} + \nabla^2_{\mathbf{x}_2})\tau \\
&= \nabla^2_{\mathbf{x}_0} g_{\mu_{ne}}(x_{0\alpha}) + \nabla^2_{\mathbf{x}_0} g_{\mu_{ne}}(x_{0\beta}) + \nabla^2_{\mathbf{x}_0} h_{\mu_{ee}}(x_{01}) + \nabla^2_{\mathbf{x}_0} h_{\mu_{ee}}(x_{02}) \\
&\quad + \nabla^2_{\mathbf{x}_1} g_{\mu_{ne}}(x_{1\alpha}) + \nabla^2_{\mathbf{x}_1} g_{\mu_{ne}}(x_{1\beta}) + \nabla^2_{\mathbf{x}_1} h_{\mu_{ee}}(x_{01}) + \nabla^2_{\mathbf{x}_1} h_{\mu_{ee}}(x_{12}) \\
&\quad + \nabla^2_{\mathbf{x}_2} g_{\mu_{ne}}(x_{2\alpha}) + \nabla^2_{\mathbf{x}_2} g_{\mu_{ne}}(x_{2\beta}) + \nabla^2_{\mathbf{x}_2} h_{\mu_{ee}}(x_{02}) + \nabla^2_{\mathbf{x}_2} h_{\mu_{ee}}(x_{12}).
\end{aligned} \tag{156}$$

Therefore, we need to evaluate the terms on the right side of Eqn. (156) one by one. Take the first one as an example, in Cartesian coordinates, the $\nabla^2_{\mathbf{x}_0} g_{\mu_{ne}}(x_{0\alpha})$ can be expanded into

$$\nabla^2_{\mathbf{x}_0} g_{\mu_{ne}}(x_{0\alpha}) = (\nabla^2_{x_0} + \nabla^2_{y_0} + \nabla^2_{z_0}) g_{\mu_{ne}}(x_{0\alpha}). \tag{157}$$

To address this problem, strategy used in Appendix (A.1) will be applied here. For $\nabla^2_{x_0} g_{\mu_{ne}}(x_{0\alpha})$, we have

$$\nabla^2_{x_0} g_{\mu_{ne}}(x_{0\alpha}) = \frac{\mathrm{d}^2}{\mathrm{d}{x_0}^2} g_{\mu_{ne}}(x_{0\alpha}). \tag{158}$$

Considering $x_{0\alpha} = \sqrt{(x_0 - x_\alpha)^2 + (y_0 - y_\alpha)^2 + (z_0 - z_\alpha)}$, the expression above can be written into

$$\frac{\mathrm{d}^2}{\mathrm{d}{x_0}^2} g_{\mu_{ne}}(x_{0\alpha}) = \frac{\mathrm{d}^2 g_{\mu_{ne}}}{\mathrm{d}x^2_{0\alpha}} \left(\frac{x_0 - x_\alpha}{x_{0\alpha}}\right)^2 + \frac{\mathrm{d}g_{\mu_{ne}}(x_{0\alpha})}{\mathrm{d}x_{0\alpha}} \frac{(y_0 - y_\alpha)^2 (z_0 - z_\alpha)^2}{x^3_{0\alpha}}. \tag{159}$$

After we get the similar expression for second derivative over $y_{0\alpha}$ and $z_{0\alpha}$ respectively, we can sum them together to get

$$\nabla^2_{\mathbf{x}_0} g_{\mu_{ne}}(x_{0\alpha}) = \frac{\mathrm{d}^2 g_{\mu_{ne}}(x_{0\alpha})}{\mathrm{d}x^2_{0\alpha}} + \frac{2}{x_{0\alpha}} \frac{\mathrm{d}g_{\mu_{ne}}(x_{0\alpha})}{\mathrm{d}x_{0\alpha}} \tag{160}$$

Repeating the tedious thing on the rest of the terms in Eqn. (156) and considering symmetry like $\nabla_{\mathbf{x}_1} h_{\mu_{ee}}(x_{01}) = \nabla_{\mathbf{x}_0} h_{\mu_{ee}}(x_{01})$, we can find

$$\begin{aligned}(\nabla^2_{\mathbf{x}_0} + \nabla^2_{\mathbf{x}_1} + \nabla^2_{\mathbf{x}_2})\tau = &\sum_{i=0}^{2}\sum_{k=\alpha}^{\beta}\left(\frac{\mathrm{d}^2}{\mathrm{d}x_{ik}^2} g_{\mu_{ne}}(x_{ik}) + \frac{2}{x_{ik}}\frac{\mathrm{d}g_{\mu_{ne}}(x_{ik})}{\mathrm{d}x_{ik}}\right) \\ &+ 2\sum_{i=0}^{2}\sum_{j>i}^{2}\left(\frac{\mathrm{d}^2}{\mathrm{d}x_{ij}^2} h_{\mu_{ee}}(x_{ij}) + \frac{2}{x_{ij}}\frac{\mathrm{d}h_{\mu_{ee}}(x_{ij})}{\mathrm{d}x_{ij}}\right).\end{aligned} \tag{161}$$

It is easy to see Eqn. (161) can be generalized by replacing the upper limit of those summations. Now, with the help of Eqn. (161), Eqn. (155) for the commutator is no longer so abstract. Instead, its general computationable form can be written as:

$$\begin{aligned}[\hat{H}, \tau] = &-\frac{1}{2}\sum_{i=0}^{\eta-1}\sum_{k=0}^{M-1}\left(\frac{\mathrm{d}^2}{\mathrm{d}x_{ik}^2} g_{\mu_{ne}}(x_{ik}) + \frac{2}{x_{ik}}\frac{\mathrm{d}g_{\mu_{ne}}(x_{ik})}{\mathrm{d}x_{ik}}\right) \\ &- \sum_{i=0}^{\eta-1}\sum_{j>i}^{\eta-1}\left(\frac{\mathrm{d}^2}{\mathrm{d}x_{ij}^2} h_{\mu_{ee}}(x_{ij}) + \frac{2}{x_{ij}}\frac{\mathrm{d}h_{\mu_{ee}}(x_{ij})}{\mathrm{d}x_{ij}}\right) \\ &+ \frac{1}{2}\sum_{i=0}^{\eta-1}\left(\sum_{\alpha=0}^{M-1} g_{\mu_{ne}}(x_{i\alpha}) + \sum_{j>i}^{\eta-1} h_{\mu_{ee}}(x_{ij})\right)\sum_{i'}^{\eta-1}\nabla^2_{\mathbf{x}_{i'}}.\end{aligned} \tag{162}$$

Now, it is time to consider $\frac{1}{2}\Big[[\hat{H}, \tau], \tau\Big]$. Once again, the first and the second row on the right side of Eqn. (162) are ordinary functions which are commutative with τ. Therefore, we have

$$\begin{aligned}\Big[[\hat{H}, \tau], \tau\Big] \to &\frac{1}{2}\sum_{i=0}^{\eta-1}\left(\sum_{\alpha=0}^{M-1} g_{\mu_{ne}}(x_{i\alpha}) + \sum_{j>i}^{\eta-1} h_{\mu_{ee}}(x_{ij})\right)\sum_{i'=0}^{\eta-1}\nabla^2_{\mathbf{x}_{i'}}\tau \\ &- \frac{1}{2}\tau\sum_{i=0}^{\eta-1}\left(\sum_{\alpha=0}^{M-1} g_{\mu_{ne}}(x_{i\alpha}) + \sum_{j>i}^{\eta-1} h_{\mu_{ee}}(x_{ij})\right)\sum_{i'=0}^{\eta-1}\nabla^2_{\mathbf{x}_{i'}}\end{aligned} \tag{163}$$

6.4.2 Idea from Cohen's paper

The previoud efford is in deadend since it cannot recover the operator $\nabla_i h \cdot \nabla_j$. After I read Cohen's paper [2], I realize where the problem is: I forgot about the operator property of $[\hat{H}, \tau]$. It is an operator, so it will be used on some function, say f, which means we need to deal with

$$[\hat{H}, \tau] f = \hat{H}(\tau f) - f\hat{H}\tau. \tag{164}$$

Although, in the previous section, most of the effort is wasted, one of the analysis is correct: in $\hat{H}$, only the kinetic energy (second-derivative) operator do not commute with τ, which agrees with [2]. In this paper, they also use this to explain why Eqn. (A.30) is truncated to second

order. Therefore, we need to focus on $\nabla_i^2(\tau f)$, which leads to

$$\begin{aligned}\nabla_i^2(\tau f) &= \nabla_i \cdot (\nabla_i(\tau f)) = \nabla_i \cdot (f\nabla_i\tau + \tau\nabla_i f) \\ &= f\nabla_i^2\tau + 2\nabla_i\tau \cdot \nabla_i f + \tau\nabla_i^2 f\end{aligned} \tag{165}$$

The first term in the last line in the equation above, $f\nabla_i^2\tau$, will be cancelled by $-f\hat{H}\tau$ in Eqn. (164). So, after this analysis, we can figure out what is left behind from Eqn. (164) is

$$[\hat{H}, \tau]_\_ = -\sum_i \left(\frac{1}{2}(\nabla_i^2\tau)_\_ + (\nabla_i\tau)\cdot\nabla_{i\_}\right). \tag{166}$$

Here I temporarily use "_" to take the place of the arbitrary function "f" on purpose to emphasize, first the operator property of this commutator; second, in the furture calculation, function to be operated should be filled in here. In the upcoming equations this "_" will not appear again, but I hope the reader can recognize there exist such an invisible phantom.

Since we are going to keep up to the second order, the term $(\nabla_i\tau)\cdot\nabla_{i\_}$ will survive in the next step in $\frac{1}{2}\Big[[\hat{H}, \tau], \tau\Big]$, which eventually leads to

$$\begin{aligned}\tilde{H} &= \hat{H} - \sum_i \left(\frac{1}{2}\nabla_i^2\tau + (\nabla\tau)\cdot\nabla_i + \frac{1}{2}(\nabla_i\tau)^2\right) \\ &= \hat{H} - \sum_{i<j}\tilde{\mathcal{K}}(\mathbf{x}_i, \mathbf{x}_j) - \sum_{i<j<k}\tilde{\mathcal{B}}(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k),\end{aligned} \tag{167}$$

where the two-body operator $\tilde{\mathcal{K}}$ and three-body operator $\tilde{\mathcal{B}}$ are defined as

$$\begin{aligned}\tilde{\mathcal{K}}(\mathbf{x}_i, \mathbf{x}_j) =& \frac{1}{2}\big(\nabla_i^2 u(\mathbf{x}_i, \mathbf{x}_j) + \nabla_j^2 u(\mathbf{x}_i, \mathbf{x}_j) + (\nabla_i u(\mathbf{x}_i, \mathbf{x}_j))^2 \\ &+ (\nabla_j u(\mathbf{x}_i, \mathbf{x}_j))^2\big) + (\nabla_i u(\mathbf{x}_i, \mathbf{x}_j)\cdot\nabla_i) \\ &+ (\nabla_j u(\mathbf{x}_i, \mathbf{x}_j)\cdot\nabla_j),\end{aligned} \tag{168}$$

and

$$\begin{aligned}\tilde{\mathcal{B}}(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) =& \nabla_i u(\mathbf{x}_i, \mathbf{x}_j)\cdot\nabla_i u(\mathbf{x}_i, \mathbf{x}_k) + \nabla_j u(\mathbf{x}_j, \mathbf{x}_i)\cdot\nabla_j u(\mathbf{x}_j, \mathbf{x}_k) \\ &+ \nabla_k u(\mathbf{x}_k, \mathbf{x}_i)\cdot\nabla_k u(\mathbf{x}_k, \mathbf{x}_j).\end{aligned} \tag{169}$$

Eqns. (168, 169) are equivalent to the equations (32, 33) (see Eqn. (150, 151)) in [3] with the symmetrical function $u(\mathbf{x}_i, \mathbf{x}_j)$ defined through $\tau = \sum_{i<j} u(\mathbf{x}_i, \mathbf{x}_j)$. By the word "symmetrical" we mean $u(\mathbf{x}_i, \mathbf{x}_j) = u(\mathbf{x}_j, \mathbf{x}_i)$.

6.4.3 Why Error Function is Used?

The preceeding subsection explained how we obtain the two-, three-body operators (see Eqns. (168, 169)) from the idea of transcorrelated Hamiltonian. However, there is still one

question left to be answered. As we know, the transcorrelated Hamiltonian methods promises us to eleminate the Coulomb cusp from the operatros. But the fact is that we can still find $1/r_{ij}$ through $\hat{H}$ in Eqn. (167). The ultimate salvation is in the appendix D of [6]. However, the fundamental idea has already been pointed out at the bottom of page 2. The authour mentioned in the transformed Hamiltonian $\tilde{H}$, the leading order terms in $1/r_{ij}$ is $\frac{1}{r_{ij}} + \tilde{W}[u]$, where

$$\tilde{W}[u] = -\frac{2}{r_{12}} \frac{\partial u(r_{12})}{\partial r_{12}} \tag{170}$$

is an operator in $\tilde{H}$ after the similarity transformation (see the first line in Eqn (A.30)). Follow the idea of range seperation density functional theory (RS-DFT) [17], the author let

$$\tilde{W}[u] + \frac{1}{r_{12}} = \frac{\operatorname{erf}(\mu r_{12})}{r_{12}}, \tag{171}$$

where the error function (see Appendix (B.2)) is introduced. This is equivalent to a differential equation

$$\frac{\partial u(r_{12}, \mu)}{\partial r_{12}} = \frac{1 - \operatorname{erf}(\mu r_{12})}{2}, \tag{172}$$

which gives the solution

$$u(r_{12}; \mu) = \frac{1}{2}\left(1 - \operatorname{erf}(\mu r_{12})\right) - \frac{1}{2\sqrt{\pi}\mu} e^{-(r_{12}\mu)^2}, \tag{173}$$

where the Jastrow factor $u(r_{12}; \mu)$ depends on a unique parameter μ. From here, we can see, as long as the Jastrow factor is wisely selected so that Eqn. (172) is fulfilled, the effect of $1/r_{ij}$ can be cancelled by $\tilde{W}(u)$.

The reason that the error function is used is explained by [17] and the graph therein, see Fig. (10).

$$\bar{\mathbf{H}} = \sum_{m,n=0}^{N-1} \omega'_{mn} \sum_{i=0}^{\eta-1} \prod_{b=0}^{\log N-1} \mathbf{X}^{m_b}_{i \log N+b} \mathbf{Z}^{n_b}_{i \log N+b} + \frac{1}{2} \sum_{m,p=1}^{N-1} \gamma'_{mp} \sum_{i \neq j}^{\eta-1} \prod_{b=0}^{\log N-1} \mathbf{Z}^{m_b}_{i \log N+b} \mathbf{Z}^{p_b}_{j \log N+b}, \tag{174}$$

where

$$\omega'_{mn} = \begin{cases} 0 \quad \text{if } m = n = 0, \\ \omega_{0n} + (N-1)\gamma_{0n} \quad \text{if } m = 0, n \neq 0, \\ \omega_{mn} \quad \text{otherwise.} \end{cases} \tag{175}$$

$$\gamma'_{mp} = \begin{cases} 0 \quad \text{if } m = 0 \text{ or } p = 0 \\ \gamma_{mp} \quad \text{otherwise} \end{cases} \tag{176}$$

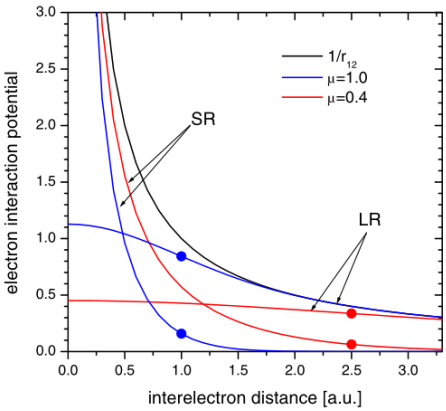

图 10: Long- and short-range electron interaction potentials, $\frac{\mathrm{erf}(\mu r_{12})}{r_{12}}$ (LR) and $\frac{1-\mathrm{erf}(\mu r_{12})}{r_{12}}$ (SR) for $\mu = 1.0$ and $\mu = 0.4$. Dots indicate position cutoff radius $r_C = 1/\mu$.

A Derivation of Formulae

A.1 Laplacian of two electron system

The following derivation is for the Eqn. (2) in [6], which is Eqn. (A.2) here. It is about the Hamiltonian of the helium atom using $r_i = |\mathbf{r}_i|$ and $r_{12} = |\mathbf{r}_1 - \mathbf{r}_2|$ coordinates

$$H = h_c + \frac{1}{r_{12}} \tag{A.1}$$

where

$$\begin{aligned} h_c = &-\frac{1}{2}\sum_{i=1}^{2}\left(\frac{\partial^2}{\partial r_i^2} + \frac{2}{r_i}\frac{\partial}{\partial r_i} + \frac{2Z}{r_i}\right) - \left(\frac{\partial^2}{\partial r_{12}^2} + \frac{2}{r_{12}}\frac{\partial}{\partial r_{12}}\right) \\ &- \left(\frac{\mathbf{r}_1}{r_1}\cdot\frac{\mathbf{r}_{12}}{r_{12}}\frac{\partial}{\partial r_1} + \frac{\mathbf{r}_2}{r_2}\cdot\frac{\mathbf{r}_{21}}{r_{21}}\frac{\partial}{\partial r_2}\right). \end{aligned} \tag{A.2}$$

Suppose $f(r)$ is a scalar function of r, then

$$\nabla\cdot(\hat{\mathbf{r}}f(r)) = \frac{\partial}{\partial x}\left(\frac{x}{r}f(r)\right) + \frac{\partial}{\partial y}\left(\frac{y}{r}f(r)\right) + \frac{\partial}{\partial z}\left(\frac{z}{r}f(r)\right). \tag{A.3}$$

Let us concentrate on $\frac{\partial}{\partial x}\left(\frac{x}{r}f(r)\right)$ first, it can be expanded into

$$\frac{\partial}{\partial x}\left(\frac{x}{r}f(r)\right) = \frac{\partial}{\partial x}\left(\frac{x}{r}\right)f(r) + \frac{x}{r}\frac{\partial}{\partial x}\left(f(r)\right). \tag{A.4}$$

Considering $r = \sqrt{x^2+y^2+z^2}$, we have $\frac{\partial}{\partial x}(\frac{x}{r}) = \frac{y^2+z^2}{r^3}$ and $\frac{\partial}{\partial x}(f(r)) = \frac{\partial f}{\partial r}\frac{\partial r}{\partial x} = \frac{x}{r}\frac{\partial f}{\partial r}$. Therefore, we obtain

$$\frac{\partial}{\partial x}\left(\frac{x}{r}f(r)\right) = \frac{y^2+z^2}{r^3}f(r) + \frac{x^2}{r^2}\frac{\partial f}{\partial r} \tag{A.5}$$

The other two terms can be handled in the same way, and we get

$$\frac{\partial}{\partial y}\left(\frac{y}{r}f(r)\right) = \frac{x^2+z^2}{r^3}f(r) + \frac{y^2}{r^2}\frac{\partial f}{\partial r} \tag{A.6}$$

and

$$\frac{\partial}{\partial z}\left(\frac{z}{r}f(r)\right) = \frac{x^2+y^2}{r^3}f(r) + \frac{z^2}{r^2}\frac{\partial f}{\partial r}. \tag{A.7}$$

From Eqn. (A.5), (A.6) and (A.7), we eventually reach

$$\nabla\cdot(\hat{\mathbf{r}}f(r)) = \frac{2}{r}f(r) + \frac{\partial f}{\partial r}. \tag{A.8}$$

Based on the coordinates we are using, we have

$$\nabla_1\cdot\nabla_1 = \nabla_1\cdot\left(\hat{\mathbf{r}}_1\frac{\partial\psi}{\partial r_1}\right) + \nabla_1\cdot\left(\hat{\mathbf{r}}_{12}\frac{\partial\psi}{\partial r_{12}}\right). \tag{A.9}$$

Using Eqn. (A.8), the first term on the right side of Eqn. (A.9) can be written into

$$\nabla_1 \cdot \left(\hat{\mathbf{r}}_1 \frac{\partial \psi(r)}{\partial r_1} \right) = \frac{2}{r_1}\psi(r) + \frac{\partial \psi(r)}{\partial r_1} \rightarrow \left(\frac{2}{r_1}\frac{\partial}{\partial r_1} + \frac{\partial^2}{\partial r_1^2} \right) \psi(r), \tag{A.10}$$

where r is used to denote the whole coordinates (r_1, r_2, r_{12}). As to the second term on the right side of Eqn. (A.9), it is

$$\begin{aligned} \nabla_1 \cdot \left(\hat{\mathbf{r}}_{12} \frac{\partial \psi}{\partial r_{12}} \right) =& \frac{\partial}{\partial x_1} \left(\frac{x_1 - x_2}{r_{12}} \frac{\partial \psi(r)}{\partial r_{12}} \right) \\ &+ \frac{\partial}{\partial y_1} \left(\frac{y_1 - y_2}{r_{12}} \frac{\partial \psi(r)}{\partial r_{12}} \right) + \frac{\partial}{\partial z_1} \left(\frac{z_1 - z_2}{r_{12}} \frac{\partial \psi(r)}{\partial r_{12}} \right) \end{aligned} \tag{A.11}$$

where $r_{12} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$, which is a function of $x_1, x_2, y_1, y_2, z_1, z_2$. Therefore, we need the chain rule to perform the derivative. Let's start with the first term on the right side of Eqn. (A.11):

$$\begin{aligned} &\frac{\partial}{\partial x_1} \left(\frac{x_1 - x_2}{r_{12}} \frac{\partial \psi}{\partial r_{12}} \right) \\ &= \frac{\partial}{\partial x_1} \left(\frac{x_1 - x_2}{r_{12}} \right) \frac{\partial \psi}{\partial r_{12}} + \frac{x_1 - x_2}{r_{12}} \frac{\partial}{\partial x_1} \left(\frac{\partial \psi}{\partial r_{12}} \right) \\ &= \left[\frac{1}{r_{12}} - \frac{(x_1 - x_2)^2}{r_{12}^3} \right] \frac{\partial \psi}{\partial r_{12}} + \frac{x_1 - x_2}{r_{12}} \left[\frac{\partial^2 \psi}{\partial r_1 \partial r_{12}} \frac{\partial r_1}{\partial x_1} + \frac{\partial^2 \psi}{\partial r_{12}^2} \frac{\partial r_{12}}{\partial x_1} \right] \\ &= \frac{(y_1 - y_2)^2 + (z_1 - z_2)^2}{r_{12}^3} \frac{\partial \psi}{\partial r_{12}} + \frac{x_1(x_1 - x_2)}{r_1 r_{12}} \frac{\partial^2 \psi}{\partial r_1 \partial r_{12}} + \frac{(x_1 - x_2)^2}{r_{12}^2} \frac{\partial^2 \psi}{\partial r_{12}^2}. \end{aligned} \tag{A.12}$$

Similarly, we have

$$\begin{aligned} &\frac{\partial}{\partial y_1} \left(\frac{y_1 - y_2}{r_{12}} \frac{\partial \psi(r)}{\partial r_{12}} \right) \\ &= \frac{(x_1 - x_2)^2 + (z_1 - z_2)^2}{r_{12}^3} \frac{\partial \psi}{\partial r_{12}} + \frac{y_1(y_1 - y_2)}{r_1 r_{12}} \frac{\partial^2 \psi}{\partial r_1 \partial r_{12}} + \frac{(y_1 - y_2)^2}{r_{12}^2} \frac{\partial^2 \psi}{\partial r_{12}^2} \end{aligned} \tag{A.13}$$

and

$$\begin{aligned} &\frac{\partial}{\partial z_1} \left(\frac{z_1 - z_2}{r_{12}} \frac{\partial \psi(r)}{\partial r_{12}} \right) \\ &= \frac{(x_1 - x_2)^2 + (y_1 - y_2)^2}{r_{12}^3} \frac{\partial \psi}{\partial r_{12}} + \frac{z_1(z_1 - z_2)}{r_1 r_{12}} \frac{\partial^2 \psi}{\partial r_1 \partial r_{12}} + \frac{(z_1 - z_2)^2}{r_{12}^2} \frac{\partial^2 \psi}{\partial r_{12}^2}. \end{aligned} \tag{A.14}$$

Sum up Eqns (A.12, A.13) and (A.14), we can rewrite Eqn. (A.11) into

$$\nabla_1 \cdot \left(\hat{\mathbf{r}}_{12} \frac{\partial \psi}{\partial r_{12}} \right) = \frac{2}{r_{12}} \frac{\partial \psi}{\partial r_{12}} + \hat{\mathbf{r}}_1 \cdot \hat{\mathbf{r}}_{12} \frac{\partial^2 \psi}{\partial r_1 \partial r_{12}} + \frac{\partial^2 \psi}{\partial r_{12}^2}. \tag{A.15}$$

Now, after substituting Eqn. (A.10) and Eqn. (A.15) into Eqn. (A.9), we get

$$\nabla_1 \cdot \nabla_1 \psi = \frac{2}{r_1} \frac{\partial \psi}{\partial r_1} + \frac{\partial^2 \psi}{\partial r_1^2} + \frac{2}{r_{12}} \frac{\partial \psi}{\partial r_{12}} + \hat{\mathbf{r}}_1 \cdot \hat{\mathbf{r}}_{12} \frac{\partial^2 \psi}{\partial r_1 \partial r_{12}} + \frac{\partial^2 \psi}{\partial r_{12}^2} \tag{A.16}$$

Repeat the same procedures, we can find the expansion of $\nabla_2 \cdot \nabla_2$, which is

$$\nabla_2 \cdot \nabla_2 \psi = \frac{2}{r_2}\frac{\partial \psi}{\partial r_2} + \frac{\partial^2 \psi}{\partial r_2^2} + \frac{2}{r_{12}}\frac{\partial \psi}{\partial r_{12}} + \hat{\mathbf{r}}_2 \cdot \hat{\mathbf{r}}_{12}\frac{\partial^2 \psi}{\partial r_2 \partial r_{12}} + \frac{\partial^2 \psi}{\partial r_{12}^2} \tag{A.17}$$

Join Eqn. (A.16) and Eqn. (A.17) together, we can recover the Eqn. (45a) in [8], which is

$$\begin{aligned}\nabla_1^2 \psi + \nabla_2^2 \psi = &\frac{\partial^2 \psi}{\partial r_1^2} + \frac{2}{r_1}\frac{\partial \psi}{\partial r_1} + \frac{\partial^2 \psi}{\partial r_2^2} + \frac{2}{r_2}\frac{\partial \psi}{\partial r_2} + 2\frac{\partial^2 \psi}{\partial r_{12}^2} \\ &+ \frac{4}{r_{12}}\frac{\partial \psi}{\partial r_{12}} + \hat{\mathbf{r}}_1 \cdot \hat{\mathbf{r}}_{12}\frac{\partial^2 \psi}{\partial r_1 \partial r_{12}} + \hat{\mathbf{r}}_2 \cdot \hat{\mathbf{r}}_{12}\frac{\partial^2 \psi}{\partial r_2 \partial r_{12}}.\end{aligned} \tag{A.18}$$

A.2 Exponentiate Matrix

If we have a real number x and a matrix A such that $A^2 = I$, we can prove

$$\exp(iAx) = \cos(x)I + i\sin(x)A. \tag{A.19}$$

Making use of the Taylor expansion of exponential function

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots, \tag{A.20}$$

we get

$$\begin{aligned}\exp(iAx) &= 1 + iAx + \frac{(iAx)^2}{2!} + \frac{(iAx)^3}{3!} + \frac{(iAx)^4}{4!} + \cdots \\ &= 1 + iAx - \frac{x^2}{2!}I - \frac{iAx^3}{3!} + \frac{x^4}{4!}I + \frac{iAx^5}{5!} + \cdots \\ &= \cos(x)I + i\sin(x)A.\end{aligned} \tag{A.21}$$

With Eqn. (A.19), we can easily understand the property of *rotation operators* about $\hat{x}, \hat{y}$ and $\hat{z}$:

$$R_x(\theta) \equiv e^{-i\theta X/2} = \cos\left(\frac{\theta}{2}\right) I - i\sin\left(\frac{\theta}{2}\right) X = \begin{bmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix} \tag{A.22}$$

$$R_y(\theta) \equiv e^{-i\theta Y/2} = \cos\left(\frac{\theta}{2}\right) I - i\sin\left(\frac{\theta}{2}\right) Y = \begin{bmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix} \tag{A.23}$$

$$R_z(\theta) \equiv e^{-i\theta Z/2} = \cos\left(\frac{\theta}{2}\right) I - i\sin\left(\frac{\theta}{2}\right) Z = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2}. \end{bmatrix} \tag{A.24}$$

If the rotation by θ is about a real unit vector $\hat{n} = (n_x, n_y, n_z)$, the epxpressions can be generalized into

$$R_{\hat{n}}(\theta) \equiv \exp(-i\theta\hat{n} \cdot \vec{\sigma}/2) = \cos\left(\frac{\theta}{2}\right) I - i\sin\left(\frac{\theta}{2}\right)(n_x X + n_y Y + n_z Z). \tag{A.25}$$

Now, let's talk about another topic: **Baker-Campbell-Hausdorf formula** (BCH), which is usually expressed as

$$e^X e^Y = e^Z, \tag{A.26}$$

where

$$Z = X + Y + \frac{1}{2}[X,Y] + \frac{1}{12}[X,[X,Y]] - \frac{1}{12}[Y,[X,Y]] + \cdots . \tag{A.27}$$

To prove Eqn. (A.26), we need to make use the following Taylor's expansion:

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots + (-1)^{n+1}\frac{x^n}{n} + \cdots \quad \text{with} \quad -1 < x \leq 1. \tag{A.28}$$

Applying Eqn. (A.20) onto e^X and e^Y, and multiplying them together, we get

$$\begin{aligned} e^X e^Y &= \left(1 + X + \frac{X^2}{2!} + \frac{X^3}{3!} + \frac{X^4}{4!} + \cdots\right)\left(1 + Y + \frac{Y^2}{2!} + \frac{Y^3}{3!} + \frac{Y^4}{4!} + \cdots\right) \\ &= 1 + Y + X + \frac{Y^2}{2!} + XY + \frac{X^2}{2!} + \frac{Y^3}{3!} + \frac{XY^2}{2!} + \frac{X^2Y}{2!} + \frac{X^3}{3!} + \frac{Y^4}{4!} + \frac{XY^3}{3!} \\ &+ \frac{X^2Y^2}{2!2!} + \frac{X^3}{3!}Y + \frac{X^4}{4!} + \cdots . \end{aligned} \tag{A.29}$$

If we only care about the order only to three, we can let $x = Y + X + \frac{Y^2}{2!} + XY + \frac{X^2}{2!} + \frac{Y^3}{3!} + \frac{XY^2}{2!} + \frac{X^2Y}{2!} + \frac{X^3}{3!}$ so that we can use Eqn. (A.28) to figure out the result of expanding $\ln(e^X e^Y)$ to the power of three. After some tedious algebraic processes, we can recover the result identical to Eqn. (A.27).

However, the Eqn. (29) in [3] is NOT directly from the BCH formula presented here. It is obtained from the same idea we obtain the BCH: expansion of the exponential function to the order we need (here we approximate to the order of 2 for τ).

$$\begin{aligned} \tilde{H} &\coloneqq e^{-\tau}\hat{H}e^{\tau} \\ &= \left(1 - \tau + \frac{\tau^2}{2!}\right)\hat{H}\left(1 + \tau + \frac{\tau^2}{2}\right) \\ &= \hat{H} + \hat{H}\tau + \hat{H}\frac{\tau^2}{2} - \tau\hat{H} - \tau\hat{H}\tau + \frac{\tau^2}{2}\hat{H} \\ &= \hat{H} + [\hat{H},\tau] + \frac{1}{2}\left[[\hat{H},\tau],\tau\right]. \end{aligned} \tag{A.30}$$

B Special Functions

This appendix collects special functions used in the note.

B.1 Chebyshev Polynomials

The *first kind of Chebyshev polynomial* over the real interval $[-1;1]$ is defined as

$$\mathbf{T}_j(x) \coloneqq \cos(j \arccos(x)). \tag{B.1}$$

Let $\theta = \arccos(x)$, note that

$$\cos(j\theta) = \sum_{\substack{0\le k\le j \\ k \text{ is even}}} \binom{j}{k} i^k \cos^{j-k}(\theta)(1-\cos^2(\theta))^{k/2}, \tag{B.2}$$

we can see Eqn. (B.1) can be written into

$$\mathbf{T}_j(x) = \sum_{\substack{0\le k\le j \\ k \text{ is even}}} \binom{j}{k} i^k x^{j-k}(1-x^2)^{k/2}, \tag{B.3}$$

with which we can see $\mathbf{T}_j(x)$ is indeed a polynomial. By the way, to prove Eqn. (B.2), we need to realize, according to Euler's equation, we have

$$\cos(j\theta) = \Re[(\cos\theta + i\sin\theta)^j]. \tag{B.4}$$

Now, let's consider the power series $\sum_{j=0}^{\infty} \mathbf{T}_j(x)y^j$ generated by the Chebyshev polynomial. Assumming $|y| < 1$, we have

$$\begin{aligned} \sum_{j=0}^{\infty} \mathbf{T}_j(x)y^j &= \sum_{j=0}^{\infty} \cos(j\theta)y^j = \sum_{j=0}^{\infty} \frac{e^{ij\theta+} + e^{-ij\theta}}{2} y^j \\ &= \frac{\frac{1}{1-ye^{i\theta}} + \frac{1}{1-ye^{-i\theta}}}{2} = \frac{1-yx}{1+y^2-2yx} \end{aligned} \tag{B.5}$$

Sometimes, we use rescaled first Chebyshev polynomial by a factor of $1/2$:

$$\tilde{\mathbf{T}}_j(x) = \begin{cases} \mathbf{T}_j(x), & j \ge 1, \\ \frac{1}{2}\mathbf{T}_0(x), & j = 0. \end{cases} \tag{B.6}$$

In this case, the alternative generative function is

$$\sum_{j=0}^{\infty} \tilde{\mathbf{T}}_j(x)y^j = \frac{1}{2} + \sum_{j=1}^{\infty} \mathbf{T}_j(x)y^j = \frac{1-y^2}{2(1+y^2-2yx)}. \tag{B.7}$$

The *Chebyshev polynomial of the second kind* is defined as

$$\mathbf{U}_j(x) \coloneqq \frac{\sin((j+1)\arccos(x))}{\sin(\arccos(x))}, \tag{B.8}$$

which has the generating function for $|y| < 1$

$$\sum_{j=0}^{\infty} \mathbf{U}_j(x) y^j = \frac{1}{1+y^2-2yx}. \tag{B.9}$$

The two kinds of Chebyshev polynomial are related as the solutions of the Pell equation

$$\mathbf{T}_j^2(x) - (x^2-1)\mathbf{U}_{j-1}^2(x) = 1. \tag{B.10}$$

The orthogonal relation between the Chebyshev polynomials is

$$\int_{-1}^{1} \mathbf{T}_j(x)\mathbf{T}_k(x)\frac{dx}{\sqrt{1-x^2}} = \begin{cases} 0, & j \neq k, \\ \frac{\pi}{2}, & j = k \neq 0, \\ \pi, & j = k = 0. \end{cases} \tag{B.11}$$

Therefore, if a function has a Chebyshev expasion

$$f(x) = \sum_{j=0}^{\infty} \beta_j \mathbf{T}_j(x), \tag{B.12}$$

the expansion coefficients must be uniquely determined by

$$\beta_j = \begin{cases} \frac{2}{\pi}\int_{-1}^{1} \mathbf{T}_j(x) f(x) \frac{dx}{\sqrt{1-x^2}}, & j \geq 1, \\ \frac{1}{\pi}\int_{-1}^{1} \mathbf{T}_0(x) f(x) \frac{dx}{\sqrt{1-x^2}}, & j = 0. \end{cases} \tag{B.13}$$

B.2 Error Function

The following content are mainly from Wikipedia. In mathematics, the **error function** (also called the **Gauss error function**), often denoted by **erf**, is a function $\mathrm{erf} : \mathbb{C} \to \mathbb{C}$ defined as:

$$\mathrm{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^{\pi} e^{-t^2} dt. \tag{B.14}$$

In general, this integral is a complex contour integral which is path-independent because $\exp(-t^2)$ is holomorphic on the whole complex plane $\mathbb{C}$. A holomorphic function is a complex-valued function of one or more complex variables that is complex differentiable in a neighbourhood of each point in a domain in complex coordinates space $\mathbb{C}^n$. In many applications, the function argument is a real number, in which case the function value is also real. This

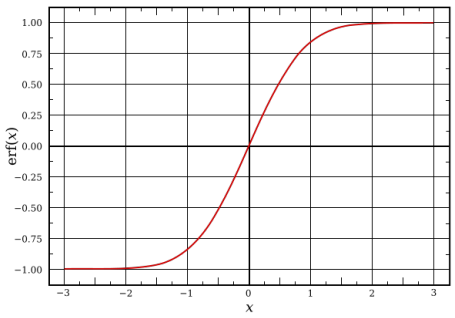

图 11: Error Function

nonelementary integral is a sigmoid function that occurs ofterm in probability, statistics, and partial differential equations.

In statistics, for non-negative real values of x, the error function has the following interpretation: for a real random variable Y which is normally distributed with mean 0 and standard deviation $\frac{1}{\sqrt{2}}$, $\operatorname{erf}(x)$ is the probability that Y falls in the range $[-x, x]$.

Two closely related functions are the **complementary error function** $\operatorname{erfc} : \mathbb{C} \to \mathbb{C}$ is defined as

$$\operatorname{erfc}(z) = 1 - \operatorname{erf}(z), \tag{B.15}$$

and the **imaginary error function** $\operatorname{erfi} : \mathbb{C} \to \mathbb{C}$ is defined as

$$\operatorname{erfi}(z) = -i \operatorname{erf}(iz). \tag{B.16}$$

Its derivative is

$$\frac{d}{dz} \operatorname{erf}(z) = \frac{2}{\sqrt{\pi}} e^{-z^2}. \tag{B.17}$$

Its antiderivative is

$$\int \operatorname{erf}(z) dz = z \operatorname{erf}(z) + \frac{e^{-z^2}}{\sqrt{\pi}} + C. \tag{B.18}$$

Its Taylor series expansion is

$$\operatorname{erf}(z) = \frac{2}{\sqrt{\pi}} \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} \frac{z^{2n+1}}{n!}. \tag{B.19}$$

C LaTeX 数学符号速查表

类别	命令	效果	
横线	`\bar{x}`	$\bar{x}$	
	`\overline{AB}`	$\overline{AB}$	
波浪线	`\tilde{x}`	$\tilde{x}$	
	`\widetilde{AB}`	$\widetilde{AB}$	
帽子	`\hat{x}`	$\hat{x}$	
	`\widehat{AB}`	$\widehat{AB}$	
向量	`\vec{v}`	$\vec{v}$	
	`\bm{v}`	$\boldsymbol{v}$	
张量积	`\otimes`	$\otimes$	
	`\bigotimes_{i=1}^n V_i`	$\bigotimes_{i=1}^{n} V_i$	
直和	`\oplus`	$\oplus$	
	`\bigoplus_{i=1}^n V_i`	$\bigoplus_{i=1}^{n} V_i$	
楔积	`\wedge`	$\wedge$	
	`\bigwedge_{i=1}^n \omega_i`	$\bigwedge_{i=1}^{n} \omega_i$	
定义符号	`\coloneqq`	$\coloneqq$	
括号	`\left\{...\right\}`	$\{...\}$	
	`\Bigg\{...\Bigg\}`	$\Bigg\{ ... \Bigg\}$	
左尖括号	`\langle`	$\langle$	
花体 O	`\mathcal{O}`	$\mathcal{O}$	
范数	`|x|`	$\|x\|$	
	`\left| \sum_{i=1}^n x_i \right|_2`	$\left\| \sum_{i=1}^{n} x_i \right\|_2$	
上确界	`\sup`	$\sup$	
下确界	`\inf`	$\inf$	
上方括号	`\lfloor x \rfloor`	$\lfloor x \rfloor$	
下方括号	`\lceil x \rceil`	$\lceil x \rceil$	
分段函数	`\begin{cases} ...`	$f(x) = \begin{cases} x^2 & \text{当} x \geq 0 \\ -x & \text{当} x < 0 \end{cases}$	
	`\end{cases}`		

Continued on nex

To be continued

类别	命令	效果	
方括号矩阵	`\begin{bmatrix} ... \end{bmatrix}`	$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$	
分段长公式	`\begin{split}...\end{split}`	$x = ... \\ = ...$	
水平省略号	`\cdots`	$\cdots$	
垂直省略号	`\vdots`	$\vdots$	
对角省略号	`\ddots`	$\ddots$	
自动省略号	`\dots`	$\dots$	
加法省略号	`\dotsb`	$a_1 + a_2 + \cdots + a_n$	
逗号省略号	`\dotsc`	$a_1, a_2, \dots, a_n$	
乘法省略号	`\dotsm`	$a_1 a_2 \cdots a_n$	
积分省略号	`\dotsi`	$\int \cdots \int$	
其他省略号	`\dotso`	$\dots$	
无序列表	`\begin{itemize} ...\end{itemize}`	• 项目一 • 项目二	
有序列表	`\begin{enumerate} ...\end{enumerate}`	1. 项目一 2. 项目二	
嵌套列表	`itemize` 或 `enumerate` 嵌套使用	多层 bullet 或编号	
自定义编号	`\usepackage{enumitem}` + `label=\alph*)` 等	a) 项目一 b) 项目二	
透明占位符	`\phantom{`内容`}` 或 `~`	空白但占位	
结构性注释（下方）	`\underbrace{`表达式`}_{{`注释`}}`	$\underbrace{a+b+c}_{\text{总项}}$	

Continued on nex

To be continued

类别	命令	效果	
结构性注释（上方）	\overbrace{表达式}{{注释}}	$\overbrace{a+b+c}^{\text{总项}}$	
数学右箭头	\rightarrow	$\rightarrow$	
数学左箭头	\leftarrow	$\leftarrow$	
双向箭头	\leftrightarrow	$\leftrightarrow$	
长右箭头	\longrightarrow	$\longrightarrow$	
长左箭头	\longleftarrow	$\longleftarrow$	
长双向箭头	\longleftrightarrow	$\longleftrightarrow$	
右双箭头	\Rightarrow	$\Rightarrow$	
左双箭头	\Leftarrow	$\Leftarrow$	
双向双箭头	\Leftrightarrow	$\Leftrightarrow$	
向上箭头	\uparrow	$\uparrow$	
向下箭头	\downarrow	$\downarrow$	
上下箭头	\updownarrow	$\updownarrow$	
双向上下箭头	\Updownarrow	$\Updownarrow$	
带标签箭头	\xrightarrow{标签}	$\xrightarrow{f}$	
带标签左箭头	\xleftarrow{标签}	$\xleftarrow{g}$	
带圈符号（基础）	\odot	$\odot$	小圆圈中有点（你最常用的）
带圈符号（基础）	\oplus	$\oplus$	圆圈加号，直和、XOR
带圈符号（基础）	\ominus	$\ominus$	圆圈减号
带圈符号（基础）	\otimes	$\otimes$	张量积，Kronecker 积
带圈符号（基础）	\oslash	$\oslash$	圆圈除号
带圈符号（基础）	\circ	$\circ$	小圆圈，函数复合
带圈符号（基础）	\bigcirc	$\bigcirc$	大圆圈
带圈符号（扩展）	\circledast	$\circledast$	圆圈星号
带圈符号（扩展）	\circledcirc	$\circledcirc$	双圈
带圈符号（扩展）	\circleddash	$\circleddash$	圆圈短横
文本带圈符号	\textcircled{A}	Ⓐ	文本模式下的带圈字符
自定义带圈符号	\tikz \node[circle,draw]{...};		可放任意符号，完全自定义

推荐宏包： \usepackage{amsmath, amssymb, bm, mathtools}

References

[1] Aleksandr Berezutskii et al. "Tensor Networks for Quantum Computing". In: *arXiv preprint* (2025). arXiv: 2503.08626 [quant-ph].

[2] Aron J. Cohen et al. "Similarity transformation of the electronic Schrödinger equation via Jastrow factorization". In: *The Journal of Chemical Physics* 151.6 (Aug. 2019), p. 061101. DOI: 10.1063/1.5116024. eprint: https://pubs.aip.org/aip/jcp/article-pdf/doi/10.1063/1.5116024/14698121/061101_1_online.pdf. URL: https://doi.org/10.1063/1.5116024.

[3] César Feniou et al. *Real-Space Chemistry on Quantum Computers: A Fault-Tolerant Algorithm with Adaptive Grids and Transcorrelated Extension*. 2025. arXiv: 2507.20583 [quant-ph]. URL: https://arxiv.org/abs/2507.20583.

[4] Timothy N Georges et al. "Pauli decomposition via the fast Walsh-Hadamard transform". In: *New Journal of Physics* 27.3 (Feb. 2025), p. 033004. ISSN: 1367-2630. DOI: 10.1088/1367-2630/adb44d. URL: http://dx.doi.org/10.1088/1367-2630/adb44d.

[5] Seyed Ehsan Ghsempoury, Gehard W. Dueck, and Stjin De Baerdemacker. "Modular cluster circuits for the Variational Quantum Eigensolver". In: *arXiv: 2305.04425v3* (2023).

[6] Emmanuel Giner. "A new form of transcorrelated Hamiltonian inspired by range-separated DFT". In: *arXiv preprint arXiv:2101.09944* (2021). URL: https://arxiv.org/abs/2101.09944.

[7] Jack D. Hidary. *Quantum Computing: An Applied Approach*. Cham, Switzerland: Springer International Publishing, 2019. ISBN: 978-3-030-23921-3. DOI: 10.1007/978-3-030-23922-0. URL: https://link.springer.com/book/10.1007/978-3-030-23922-0.

[8] Egil A. Hylleraas. "The Schrödinger Two-Electron Atomic Problem". In: ed. by Per-Olov Löwdin. Vol. 1. Advances in Quantum Chemistry. Academic Press, 1964, pp. 1–33. DOI: https://doi.org/10.1016/S0065-3276(08)60373-1. URL: https://www.sciencedirect.com/science/article/pii/S0065327608603731.

[9] IBM Quantum. *Quantum Chemistry with VQE*. https://qiskit.qotlabs.org/learning/courses/quantum-diagonalization-algorithms/vqe. Accessed: 2026-03-13. 2024.

[10] IBM Quantum Team. *Sample-based quantum diagonalization of a chemistry Hamiltonian*. https://quantum.cloud.ibm.com/docs/en/tutorials/sample-based-quantum-diagonalization. Accessed: 2025-08-01. 2024.

[11] Abhijith J. et al. “Quantum Algorithm Implementations for Beginners”. Version v3. In: *arXiv preprint arXiv:1804.03719* (2022). URL: https://arxiv.org/abs/1804.03719.

[12] R Jastrow. “Many-Body Problem with Strong Forces”. In: *Phys. Rev* 98 (1955).

[13] Robert Loredo. *Learn Quantum Computing with Python and IBM Quantum Experience*. Levery Place, 35 Livery Street, Birmingham, UK: Packt Publishing Ltd., 2020. ISBN: 978-1-83898-100-6.

[14] Guang Hao Low and Yuan Su. “Quantum Eigenvalue Processing”. In: *Proceedings of the 65th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. Oct. 2024, pp. 1051–1062. DOI: 10.1109/FOCS61266.2024.00070. URL: https://www.researchgate.net/publication/386261589_Quantum_Eigenvalue_Processing.

[15] Mario Motta et al. “Bridging physical intuition and hardware efficiency for correlated electronic states: the local unitary cluster Jastrow ansatz for electronic structure”. In: *Chemical Science* 14 (2023), p. 11213. DOI: 10.1039/d3sc02516k. URL: https://pubs.rsc.org/en/content/articlepdf/2023/sc/d3sc02516k.

[16] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary Edition. Cambridge, UK: Cambridge University Press, 2010. ISBN: 978-1-107-00217-3. DOI: 10.1017/CBO9780511976667. URL: https://doi.org/10.1017/CBO9780511976667.

[17] Katarzyna Pernal and Michał Hapka. “Range-separated multiconfigurational density functional theory methods”. In: *WIREs Computational Molecular Science* 12.2 (2022), e1566. DOI: https://doi.org/10.1002/wcms.1566. eprint: https://wires.onlinelibrary.wiley.com/doi/pdf/10.1002/wcms.1566. URL: https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/wcms.1566.

[18] Alberto Peruzzo et al. “A variational eigenvalue solver on a photonic quantum processor”. In: *Nature Communications* 5 (2014), p. 4213. DOI: 10.1038/ncomms5213.

[19] Javier Robledo-Moreno et al. “Chemistry beyond the scale of exact diagonalization on a quantum-centric supercomputer”. In: *Science Advances* 11.25 (June 2025). ISSN: 2375-2548. DOI: 10.1126/sciadv.adu9991. URL: http://dx.doi.org/10.1126/sciadv.adu9991.

[20] Changpeng Shao and Jin-Peng Liu. “Solving generalized eigenvalue problems by ordinary differential equations on a quantum computer”. In: *arXiv preprint arXiv:2010.15027* (2020). URL: https://arxiv.org/abs/2010.15027.

[21] A. Tranter et al. “The Bravyi–Kitaev Transformation: Properties and Applications”. In: *Int. J. Quantum Chem* 115 (2015), pp. 1431–1441. DOI: 10.1002/qua.24969.